Tools and Techniques for Data Science

Introduction

Agenda

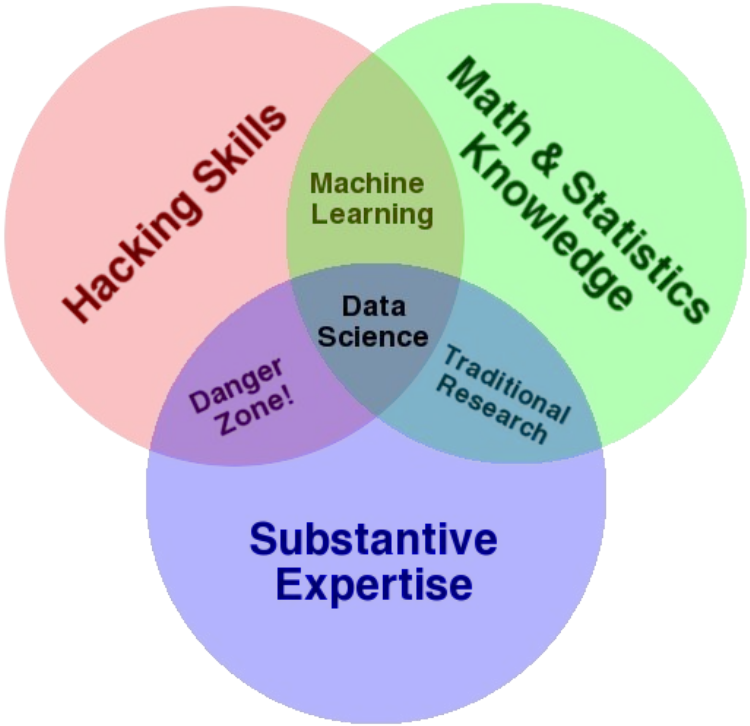
Definition of Data Science

Cloud Computing and Big Data

Tools in Data Science

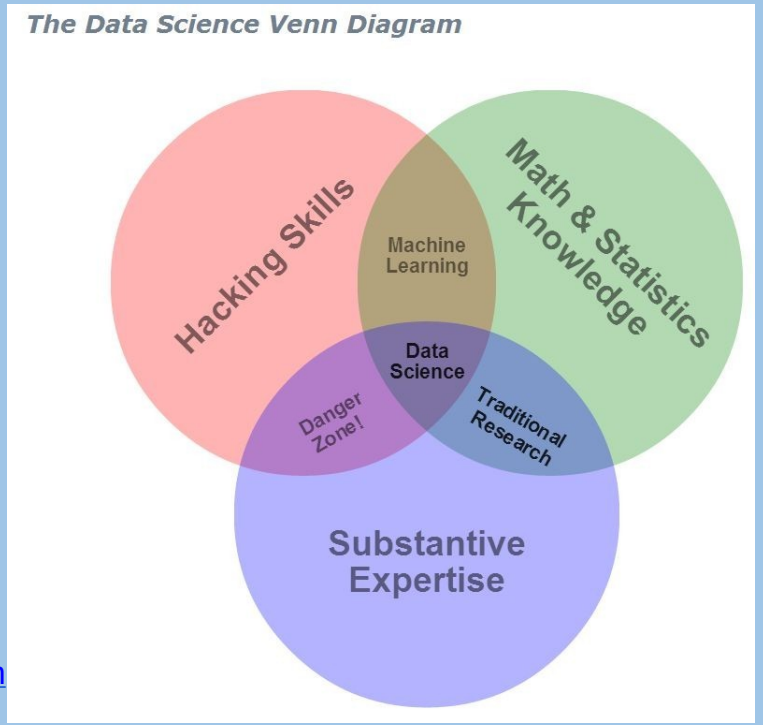
Techniques in Data Science

Definition of Data Science



What is data science ?

Hacking (Programming) + Maths/Statistics + Domain Knowledge = Data Science



http://drewconway.com/zia/2013/3/26/the-data-science-venn-diagram

What is a Data Scientist ?

a data scientist is simply a data analyst living in **california**

What is a Data Scientist

a data scientist is simply a person who can

write code

understand statistics

derive insights from data

Oh really, is this a Data Scientist ?

a data scientist is simply a person who can

write code = in R,Python,Java, SQL, Hadoop (Pig,HQL,MR) etc

= for data storage, querying, summarization, visualization

= how efficiently, and in time (fast results?)

= where on databases, on cloud, servers

and understand **enough** statistics

to derive **insights** from data

so **business** can make **decisions**

Cheat Sheets for Data Scientists

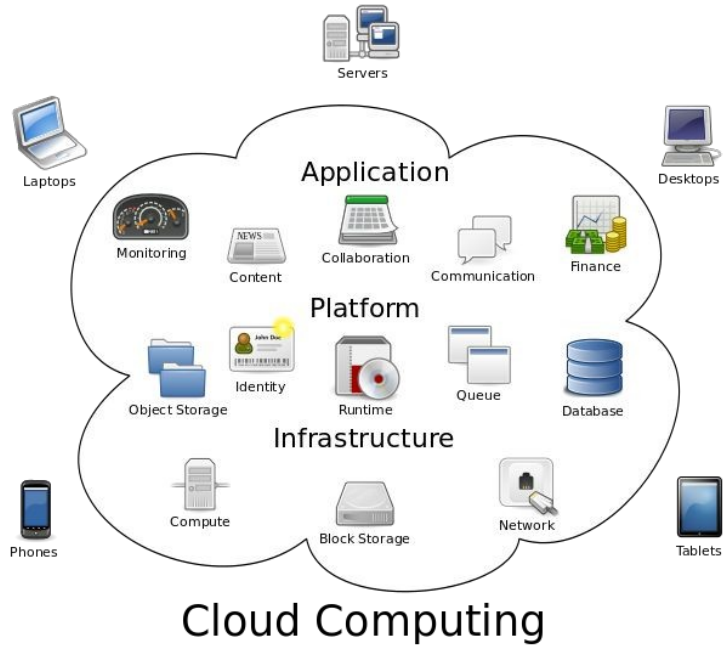
http://www.kdnuggets.com/2014/05/guide-to-data-science-cheat-sheets.html

a data scientist is simply a person who can write code in a few languages (primarily R, Python and SQL) for data querying, manipulation , aggregation, and visualization using enough statistical knowledge to give back actionable insights to the business for making decisions.

Since this rather practical definition of a data scientist is reinforced by the accompanying words on a job website for “data scientists” , ergo, here are some tools for learning the primary languages in data science- Python, R and SQL. A cheat sheet or reference card is a compilation of mostly used commands to help you learn that language’s syntax at a faster rate.

Cloud Computing

1. the practice of using a network of remote servers hosted on the Internet to store, manage, and process data, rather than a local server or a personal computer.



http://csrc.nist.gov/publications/nistpubs/800-145/SP800-145.pdf

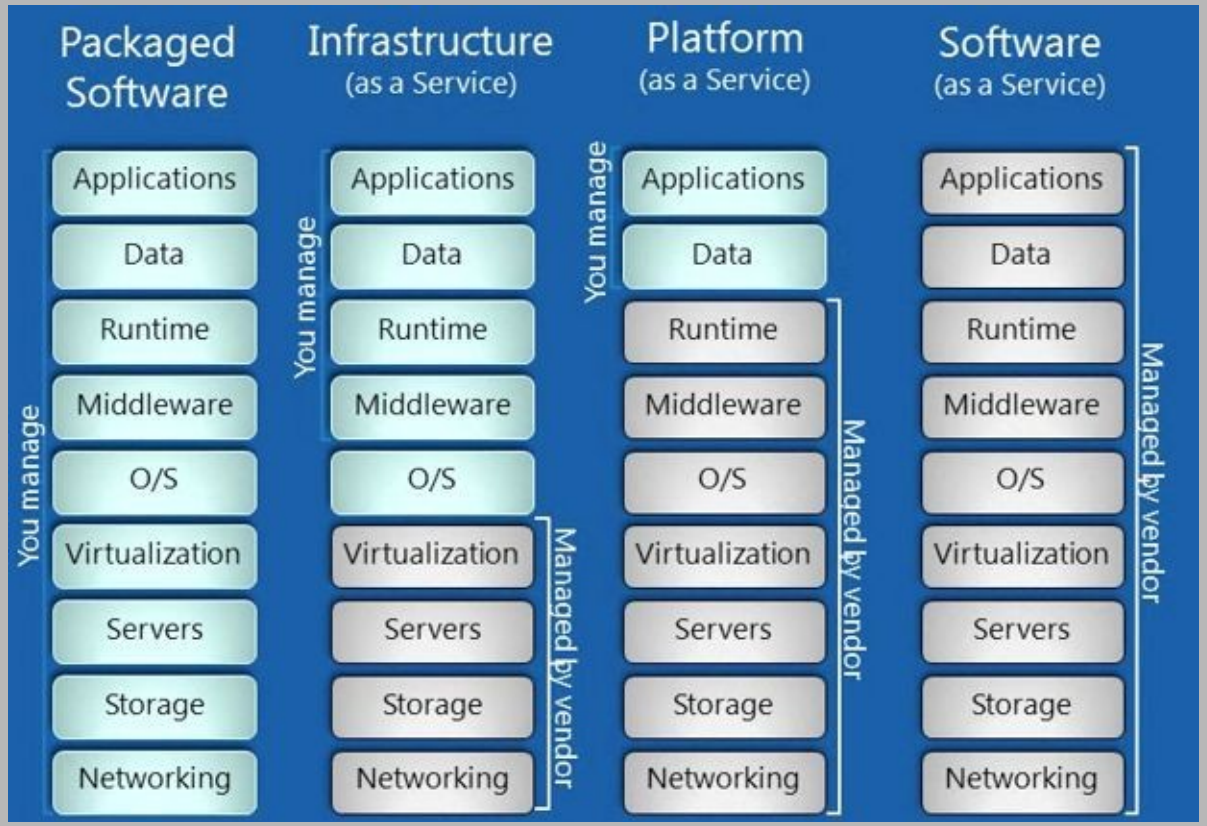
Cloud Computing

1. the practice of using a network of remote servers hosted on the Internet to store, manage, and process data, rather than a local server or a personal computer.



http://csrc.nist.gov/publications/nistpubs/800-145/SP800-145.pdf

Cloud Computing



Introduction to Big Data

extremely large data sets that may be analysed computationally to reveal patterns, trends, and associations, especially relating to human behaviour and interactions.

Big data is a term that describes the large volume of data – both structured and unstructured – that inundates a business on a day-to-day basis. But it’s not the amount of data that’s important. It’s what organizations do with the data that matters. Big data can be analyzed for insights that lead to better decisions and strategic business moves.

http://www.sas.com/en_us/insights/big-data/what-is-big-data.html

The Three Vs of Big Data

industry analyst Doug Laney articulated the now-mainstream definition of big data as the three Vs:

Volume. Organizations collect data from a variety of sources, including business transactions, social media and information from sensor or machine-to-machine data. In the past, storing it would've been a problem – but new technologies (such as Hadoop) have eased the burden.

Velocity. Data streams in at an unprecedented speed and must be dealt with in a timely manner. RFID tags, sensors and smart metering are driving the need to deal with torrents of data in near-real time.

Variety. Data comes in all types of formats – from structured, numeric data in traditional databases to unstructured text documents, email, video, audio, stock ticker data and financial transactions.

http://www.ibmbigdatahub.com/infographic/four-vs-big-data

The FOUR V's of Big Data

From traffic patterns and music downloads to web history and medical records, data is recorded, stored, and analyzed to enable the technology and services that the world relies on every day. But what exactly is big data, and how can these massive amounts of data be used?

As a leader in the sector, IBM data scientists break big data into four dimensions: **Volume, Velocity, Variety and Veracity**

Depending on the industry and organization, big data encompasses information from multiple internal and external sources such as transactions, social media, enterprise content, sensors and mobile devices. Companies can leverage data to adapt their products and services to better meet customer needs, optimize operations and infrastructure, and find new sources of revenue.

By 2015
4.4 MILLION IT JOBS
will be created globally to support big data, with 1.9 million in the United States

Volume
SCALE OF DATA

40 ZETTABYTES
[43 TRILLION GIGABYTES]
of data will be created by 2020, an increase of 300 times from 2005

2005 2020

It's estimated that
2.5 QUINTILLION BYTES
[2.3 TRILLION GIGABYTES]
of data are created each day

6 BILLION PEOPLE
have cell phones

WORLD POPULATION: 7 BILLION

Most companies in the U.S. have at least
100 TERABYTES
[100,000 GIGABYTES]
of data stored

Variety
DIFFERENT FORMS OF DATA

As of 2011, the global size of data in healthcare was estimated to be
150 EXABYTES
[161 BILLION GIGABYTES]

By 2014, it's anticipated there will be
420 MILLION WEARABLE, WIRELESS HEALTH MONITORS

4 BILLION+ HOURS OF VIDEO
are watched on YouTube each month

30 BILLION PIECES OF CONTENT
are shared on Facebook every month

400 MILLION TWEETS
are sent per day by about 200 million monthly active users

Velocity
ANALYSIS OF STREAMING DATA

The New York Stock Exchange captures
1 TB OF TRADE INFORMATION
during each trading session

Modern cars have close to
100 SENSORS
that monitor items such as fuel level and tire pressure

By 2016, it is projected there will be

Veracity
UNCERTAINTY OF DATA

1 IN 3 BUSINESS LEADERS
don't trust the information they use to make decisions

Poor data quality costs the US economy around
$3.1 TRILLION A YEAR

27% OF RESPONDENTS

in one survey were unsure of how much of their data was inaccurate

Waiting for l.sharethis.com...

Understanding Hadoop

The Apache Hadoop software library is a framework that allows for the distributed processing of large data sets across clusters of computers using simple programming models. It is designed to scale up from single servers to thousands of machines, each offering local computation and storage. Rather than rely on hardware to deliver high-availability, the library itself is designed to detect and handle failures at the application layer, so delivering a highly-available service on top of a cluster of computers, each of which may be prone to failures.

The project includes these modules:

- **Hadoop Common**: The common utilities that support the other Hadoop modules.
- **Hadoop Distributed File System (HDFS™)**: A distributed file system that provides high-throughput access to application data.
- **Hadoop YARN**: A framework for job scheduling and cluster resource management.
- **Hadoop MapReduce**: A YARN-based system for parallel processing of large data sets.

http://hadoop.apache.org/

Understanding Hadoop

- Other Hadoop-related projects at Apache include:
- **Ambari™**: A web-based tool for provisioning, managing, and monitoring Apache Hadoop clusters which includes support for Hadoop HDFS, Hadoop MapReduce, Hive, HCatalog, HBase, ZooKeeper, Oozie, Pig and Sqoop. Ambari also provides a dashboard for viewing cluster health such as heatmaps and ability to view MapReduce, Pig and Hive applications visually alongwith features to diagnose their performance characteristics in a user-friendly manner.
- **Avro™**: A data serialization system.
- **Cassandra™**: A scalable multi-master database with no single points of failure.
- **Chukwa™**: A data collection system for managing large distributed systems.
- **HBase™**: A scalable, distributed database that supports structured data storage for large tables.
- **Hive™**: A data warehouse infrastructure that provides data summarization and ad hoc querying.
- **Mahout™**: A Scalable machine learning and data mining library.
- **Pig™**: A high-level data-flow language and execution framework for parallel computation.
- **Spark™**: A fast and general compute engine for Hadoop data. Spark provides a simple and expressive programming model that supports a wide range of applications, including ETL, machine learning, stream processing, and graph computation.
- **Tez™**: A generalized data-flow programming framework, built on Hadoop YARN, which provides a powerful and flexible engine to execute an arbitrary DAG of tasks to process data for both batch and interactive use-cases. Tez is being adopted by Hive™, Pig™ and other frameworks in the Hadoop ecosystem, and also by other commercial software (e.g. ETL tools), to replace Hadoop™ MapReduce as the underlying execution engine.
- **ZooKeeper™**: A high-performance coordination service for distributed applications.

Why Learn Hadoop

Hadoop, however, is specifically designed to have a very flat scalability curve. After a Hadoop program is written and functioning on ten nodes, very little--if any--work is required for that same program to run on a much larger amount of hardware.

MapReduce: Isolated Processes

Hadoop limits the amount of communication which can be performed by the processes, as each individual record is processed by a task in isolation from one another. While this sounds like a major limitation at first, it makes the whole framework much more reliable

In a Hadoop cluster, data is distributed to all the nodes of the cluster as it is being loaded in. The Hadoop Distributed File System (HDFS) will split large data files into chunks which are managed by different nodes in the cluster. In addition to this each chunk is replicated across several machines, so that a single machine failure does not result in any data being unavailable.

https://developer.yahoo.com/hadoop/tutorial/module1.html

HDFS

HDFS, the Hadoop Distributed File System, is a distributed file system designed to hold very large amounts of data (terabytes or even petabytes), and provide high-throughput access to this information. Files are stored in a redundant fashion across multiple machines to ensure their durability to failure and high availability to very parallel applications.

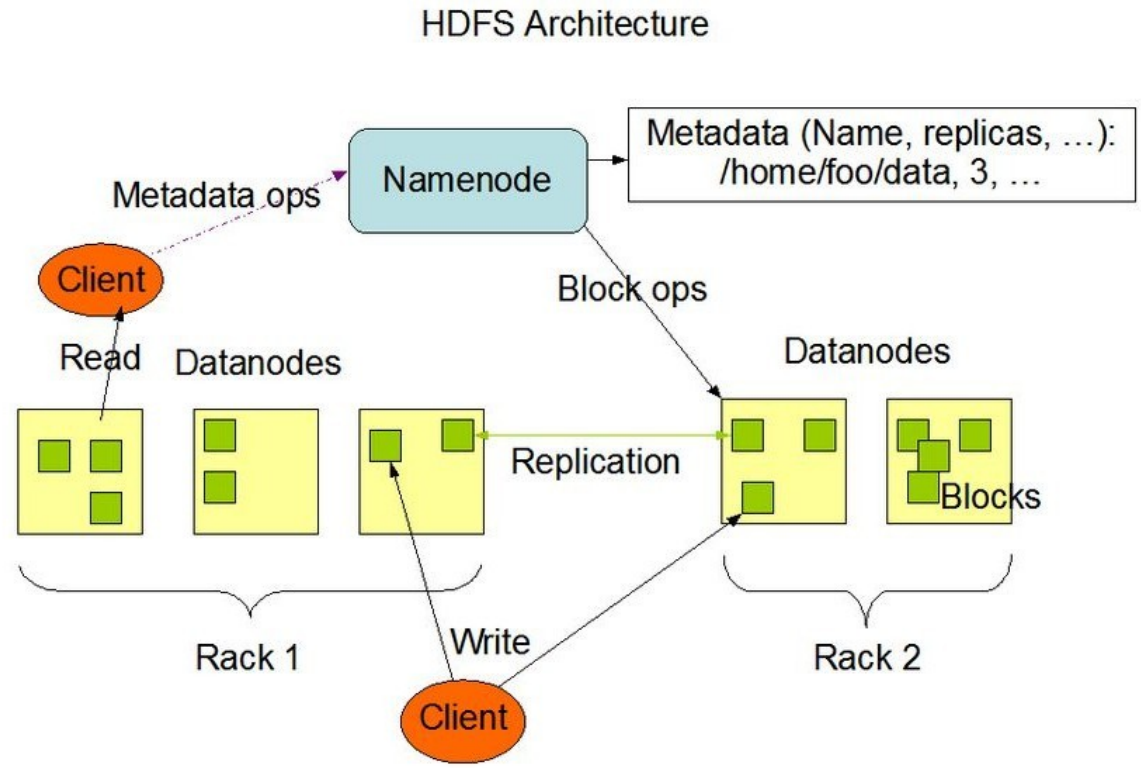
HDFS is a block-structured file system: individual files are broken into blocks of a fixed size. These blocks are stored across a cluster of one or more machines with data storage capacity. Individual machines in the cluster are referred to as **DataNodes**. A file can be made of several blocks, and they are not necessarily stored on the same machine; the target machines which hold each block are chosen randomly on a block-by-block basis. Thus access to a file may require the cooperation of multiple machines, but supports file sizes far larger than a single-machine DFS; individual files can require more space than a single hard drive could hold.

https://developer.yahoo.com/hadoop/tutorial/module2.html

HDFS

HOW HDFS WORKS

An HDFS cluster is comprised of a NameNode, which manages the cluster metadata, and DataNodes that store the data. Files and directories are represented on the NameNode by inodes. Inodes record attributes like permissions, modification and access times, or namespace and disk space quotas.



Five Daemons of Hadoop

NameNode- the Master node which is responsible for storing the meta-data for all the files and directories. It has information such as the blocks that make a file, and where are those blocks located in the cluster.

Secondary **NameNode**- It periodically merges changes in the NameNode with the edit log so that it doesn't grow too large in size. It also keeps a copy of the image which can be used in case of failure of NameNode.

DataNode - Stores actual HDFS data blocks.

JobTracker -JobTracker is the daemon service for submitting and tracking MapReduce jobs in Hadoop. There is only One Job Tracker process run on any hadoop cluster.

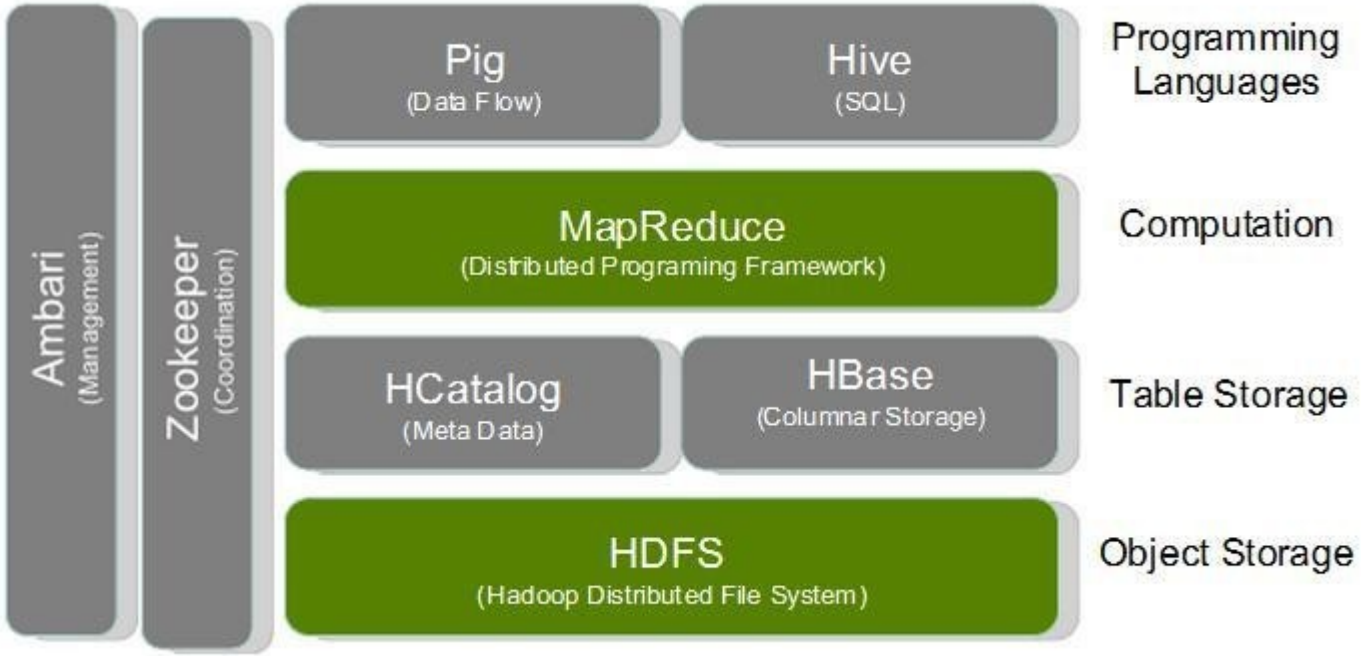
TaskTracker -A TaskTracker is a slave node daemon in the cluster that accepts tasks (Map, Reduce and Shuffle operations) from a JobTracker. There is only One Task Tracker process run on any hadoop slave node.

Following 3 Daemons run on Master nodes NameNode - Secondary NameNode - JobTracker -

Following 2 Daemons run on each Slave nodes DataNode . TaskTracker **http://www.fromdev.**

com/2010/12/interview-questions-hadoop-mapreduce.html

Big Data: Hadoop Stack



NoSQL

A NoSQL (Not-only-SQL) database is one that has been designed to store, distribute and access data using methods that differ from relational databases (RDBMS’s). NoSQL technology was originally created and used by Internet leaders such as Facebook, Google, Amazon, and others who required database management systems that could write and read data anywhere in the world, while scaling and delivering performance across massive data sets and millions of users.

NoSQL

https://www.datastax.com/nosql-databases

Use an RDBMS when you need/have…	Use NoSQL when you need/have…
Centralized applications (e.g. ERP)	Decentralized applications (e.g. Web, mobile and IOT)
Moderate to high availability	Continuous availability; no downtime
Moderate velocity data	High velocity data (devices, sensors, etc.)
Data coming in from one/few locations	Data coming in from many locations
Primarily structured data	Structured, with semi/unstructured
Complex/nested transactions	Simple transactions
Primary concern is scaling reads	Concern is to scale both writes and reads
Philosophy of scaling up for more users/data	Philosophy of scaling out for more users/data
To maintain moderate data volumes with purge	To maintain high data volumes; retain forever

Basic Statistics

Some of the basic statistics that every data scientist should know are given here. This assumes rudimentary basic knowledge of statistics (like measures of central tendency or variation) and basic familiarity with some of the terminology used by statisticians.

- **Random Sampling**- In truly random sampling,the sample should be representative of the entire data. RAndom sampling remains of relevance in the era of Big Data and Cloud Computing

- **Distributions-** A data scientist should know the distributions (normal, Poisson, Chi Square, F) and also how to determine the distribution of data.

- **Hypothesis Testing** - Hypothesis testing is meant for testing assumptions statistically regarding values of central tendency (mean, median) or variation. A good example of an easy to use software for statistical testing is the “test” tab in the Rattle GUI in R.

- **Outliers-** Checking for outliers is a good way for a data scientist to see anomalies as well as identify data quality. The box plot (exploratory data analysis) and the outlierTest function from car package (Bonferroni Outlier Test) is how statistical rigor can be maintained to outlier detection.

Basic Techniques

Some of the basic techniques that a data scientist must know are listed as follows-

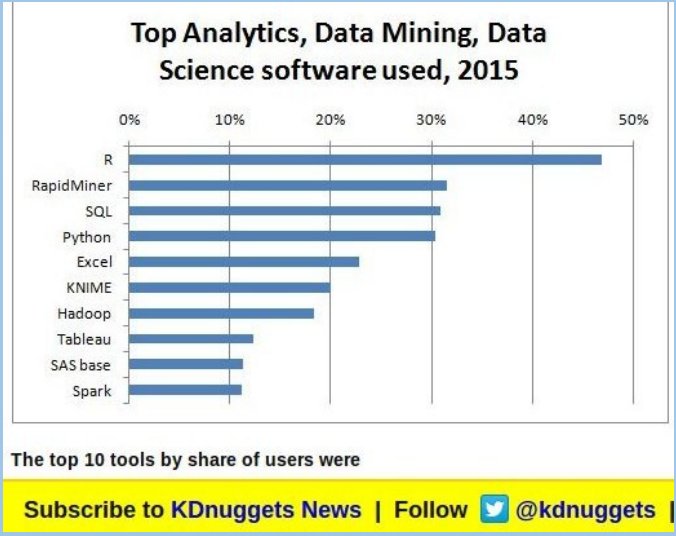
- Text Mining - In text mining , text data is analyzed for frequencies, associations and corelation for predictive purposes. The tm package from R greatly helps with text mining.
- Sentiment Analysis- In sentiment analysis the text data is classified based on a sentiment lexicography (eg which says happy is less positive than delighted but more positive than sad) to create sentiment scores of the text data mined.
- Social Network Analysis- In social network analysis, the direction of relationships, the quantum of messages and the study of nodes,edges and graphs is done to give insights..
- Time Series Forecasting- Data is said to be auto regressive with regards to time if a future value is dependent on a current value for a variable. Technqiues such as ARIMA and exponential smoothing and R packages like forecast greatly assist in time series forecasting.
- Web Analytics
- Social Media Analytics
- Data Mining or Machine Learning

Data Science Tools

- R
- Python
- Tableau
- Spark with ML
- Hadoop (Pig and Hive)
- SAS
- SQL

Data Science with R

A popular language
in Data Science



R moves up to 5th place in IEEE language rankings

IEEE Spectrum has just published its third annual ranking with its 2016 Top Programming Languages, and the R Language is once again near the top of the list, moving up one place to fifth position.

Language Rank	Types	Spectrum Ranking
1. C		100.0
2. Java		98.1
3. Python		98.0
4. C++		95.9
5. R		87.9
6. C#		86.7
7. PHP		82.8
8. JavaScript		82.2
9. Ruby		74.5
10. Go		71.9

As I said last year (when R moved up to take sixth place), this is an extraordinary result for a domain-specific language. The other four languages in the top 5 (C, Java, Python amd C++) are all general-purpose languages, suitable for just about any programming task. R by contrast is a language specifically for data science, and its high ranking here reflects both the critical importance of data science as a discipline today, and of R as the language of choice for data scientists

What Is R

https://www.r-project.org/about.html

R is an integrated suite of software facilities for data manipulation, calculation and graphical display. It includes

- an effective data handling and storage facility,
- a suite of operators for calculations on arrays, in particular matrices,
- a large, coherent, integrated collection of intermediate tools for data analysis,
- graphical facilities for data analysis and display either on-screen or on hardcopy, and
- a well-developed, simple and effective programming language which includes conditionals, loops, user-defined recursive functions and input and output facilities.

Install R

https://cran.r-project.org/bin/windows/base/

R-3.3.1 for Windows (32/64 bit)

Download R 3.3.1 for Windows (70 megabytes, 32/64 bit)

Installation and other instructions
New features in this version

If you want to double-check that the package you have downloaded exactly matches the package distributed by R, you can compare the md5sum of the .exe to the true fingerprint. You will need a version of md5sum for windows: both graphical and command line versions are available.

Frequently asked questions

- Does R run under my version of Windows?
- How do I update packages in my previous version of R?
- Should I run 32-bit or 64-bit R?

Please see the R FAQ for general information about R and the R Windows FAQ for Windows-specific information.

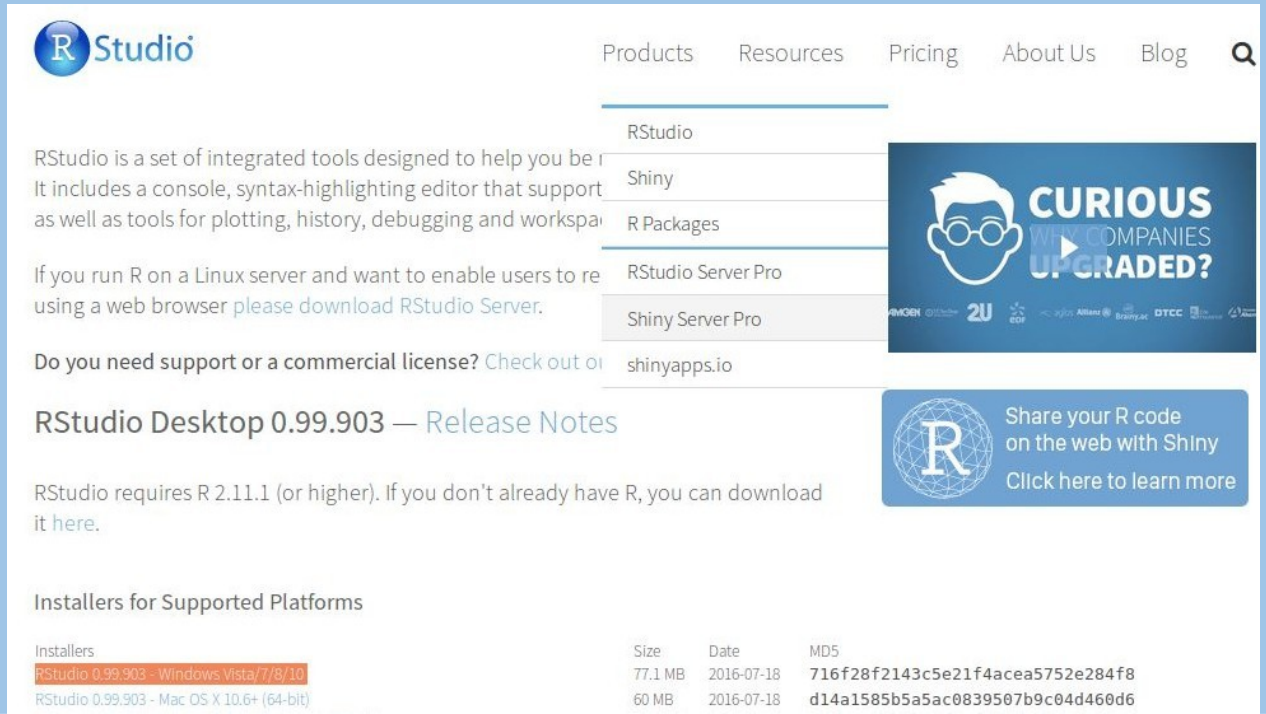
Other builds

- Patches to this release are incorporated in the r-patched snapshot build.
- A build of the development version (which will eventually become the next major release of R) is available in the r-devel snapshot build.
- Previous releases

Note to webmasters: A stable link which will redirect to the current Windows binary release is <CRAN MIRROR>/bin/windows/base/release.htm.

Install RStudio

https://www.rstudio.com/products/rstudio/download/



Statistical Software Landscape

SAS

Python (Pandas)

IBM SPSS

R

Julia

Clojure

Octave

Matlab

JMP

E

views

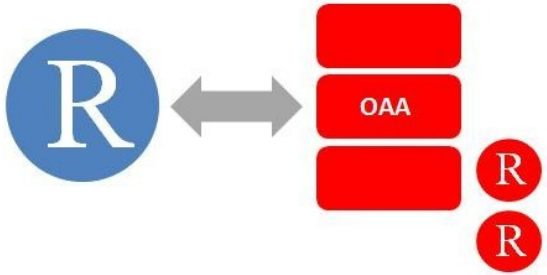


Using R with other software

https://rforanalytics.wordpress.com/useful-links-for-r/using-r-from-other-software/

Tableau http://www.tableausoftware.com/new-features/r-integration

Qlik http://qliksolutions.ru/qlikview/add-ons/r-connector-eng/



Oracle R http://www.oracle.com/technetwork/database/database-technologies/r/r-enterprise/overview/index.html

Rapid Miner https://rapid-i.com/content/view/202/206/lang,en/#r

JMP http://blogs.sas.com/jmp/index.php?/archives/298-JMP-Into-R!.html

Using R with other software

https://rforanalytics.wordpress.com/useful-links-for-r/using-r-from-other-software/

SAS/IML http://www.sas.com/technologies/analytics/statistics/iml/index.html

Teradata http://developer.teradata.com/applications/articles/in-database-analytics-with-ter

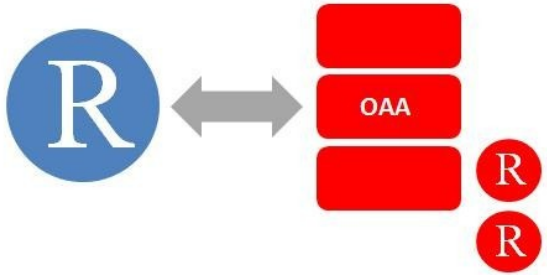
Pentaho http://bigdatatechworld.blogspot.in/2013/10/integration-of-rweka-with-pentaho-data.html

IBM SPSS

https://www14.software.ibm.com/webapp/iwm/web/signup.do?source=ibm-analytics&S_PKG=ov18855&S_TACT=M161003W&dy n form=127&lang=en_US

TIBCO TERR

http://spotfire.tibco.com/discover-spotfire/what-does-spotfire-do/predictive-analytics/tibco-enterprise-runtime-for-r-terr



Some Advantages of R

open source

free

large number of algorithms and packages esp for statistics

flexible

very good for data visualization

superb community

rapidly growing

can be used with other software



Some Disadvantages of R

in memory (RAM) usage

steep learning curve

some IT departments frown on open source

verbose documentation

tech support

evolving ecosystem for corporates



Solutions for Disadvantages of R

in memory (RAM) usage ⇒ specialized packages, in database computing

steep learning curve ⇒ TRAINING !!!

some IT departments frown on open source ⇒ TRAINING and education!

verbose documentation ⇒ CRAN View , R Documentation

tech support ⇒ expanding pool of resources

evolving ecosystem for corporates ⇒ getting better with MS et al

What is SAS?

- **SAS** (Statistical Analysis System
- Software suite developed by SAS Institute for advanced analytics, business intelligence, data management, and predictive analytics
- Developed at North Carolina State University from 1966 until 1976, when SAS Institute was incorporated.
- Further developed in the 1980s and 1990s with the additional statistical procedures and

Components of SAS

Currently, SAS has more than 200 components, some of them are –

- Base SAS – Basic procedures and data management
- SAS/STAT – Statistical analysis
- SAS/GRAPH – Graphics and presentation
- SAS/OR – Operations research
- SAS/ETS – Econometrics and Time Series Analysis
- SAS/IML – Interactive matrix language

SAS University Edition

http://www.sas.com/en_in/software/university-edition/download-software.html

How to download SAS University Edition?

1. Visit the link http://www.sas.com/en_in/software/university-edition/download-software.html

How to Download SAS® University Edition

Step 1: Verify that you have a compatible virtualization software package.

Because SAS University Edition is a virtual application (or vApp), you need virtualization software to run it. Compatible virtualization software packages are listed in the table below. If you don't already have compatible virtualization software, get the free Oracle VM VirtualBox (select release **4.3.12***).

Windows	Oracle VM VirtualBox 4.3.12*	VMware Player 6.0 or later
OS X	Oracle VM VirtualBox 4.3.12*	VMware Fusion for OS X 6.0
Linux	Oracle VM VirtualBox 4.3.12*	VMware Player for Linux 6.0 or later

* Due to known compatibility issues between SAS University Edition and the most recent releases of Oracle VM VirtualBox, you must install **release 4.3.12**. We are investigating and will update this page as soon as the issues are resolved. In the meantime, do not accept any updates suggested for VirtualBox. Thanks for your patience, and enjoy SAS University Edition!

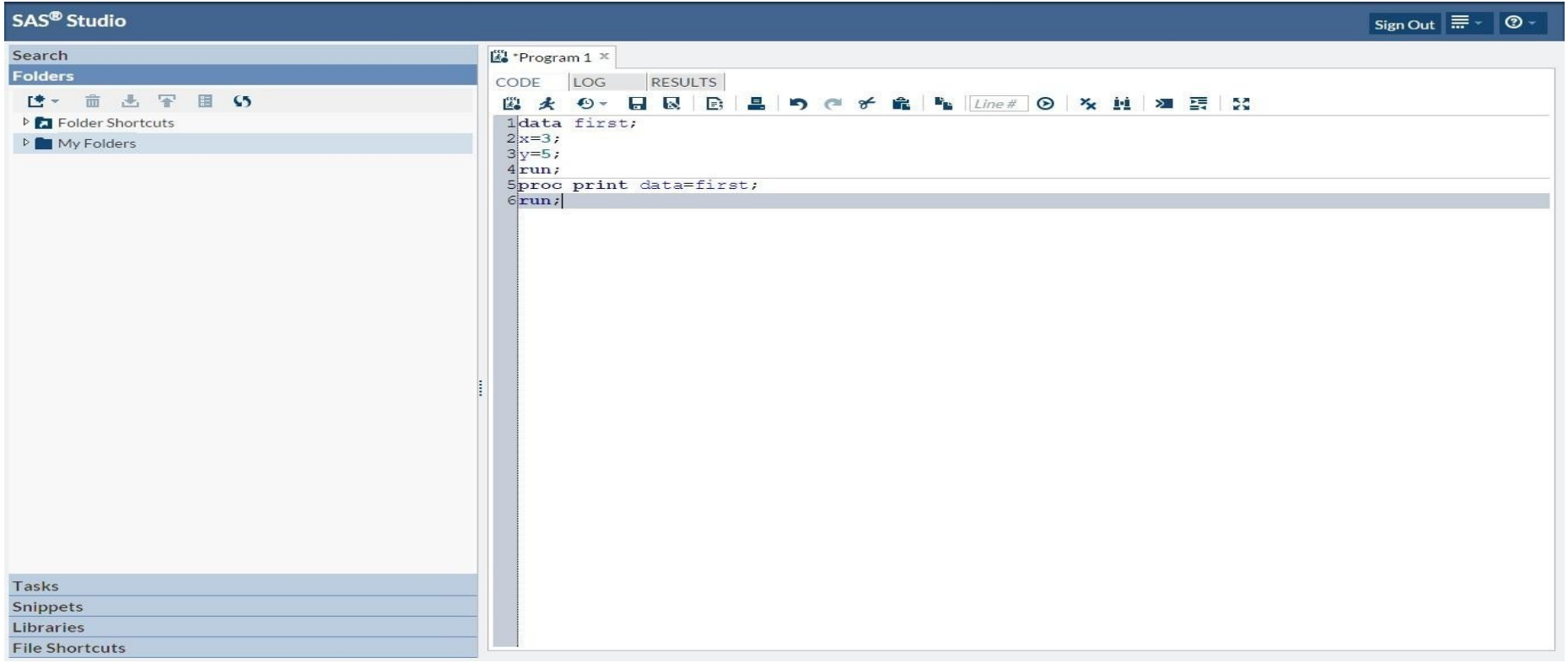
2. Because SAS University Edition is a virtual application , one needs a virtualization software to run it. So, download Oracle VM VirtualBox or VMware Player and install it on your pc/laptop.
3. Now, click on SAS® University Edition for (respective) Virtual application

SAS® University Edition for VirtualBox

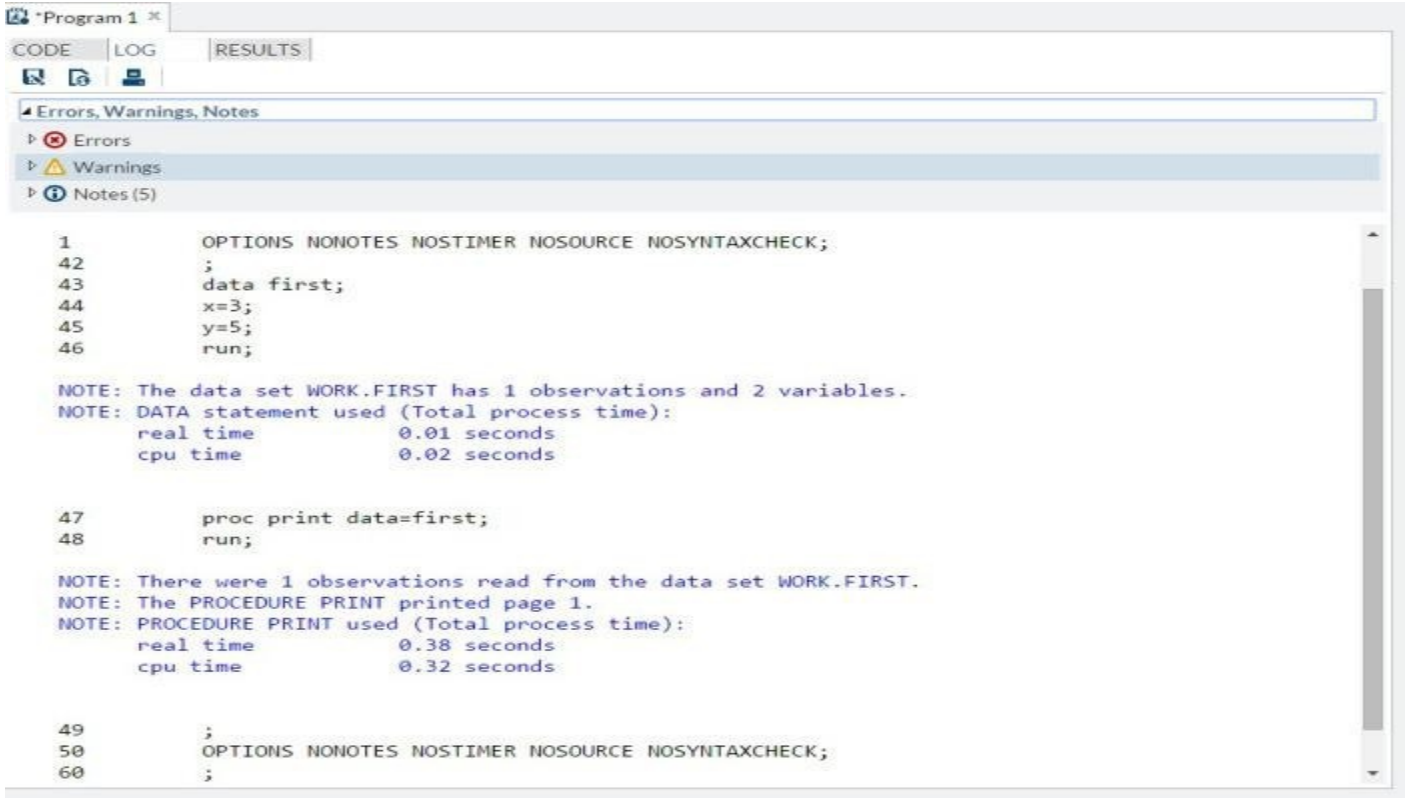
SAS Interface

- Code Editor where we write and modify mode
- Log is a record of everything that we do in SAS session or SAS program :-
  - Program statements identified by line numbers
  - Messages that begin with NOTE, INFO, WARNING, ERROR, or an error number
  - Process time
- Result displays printed output

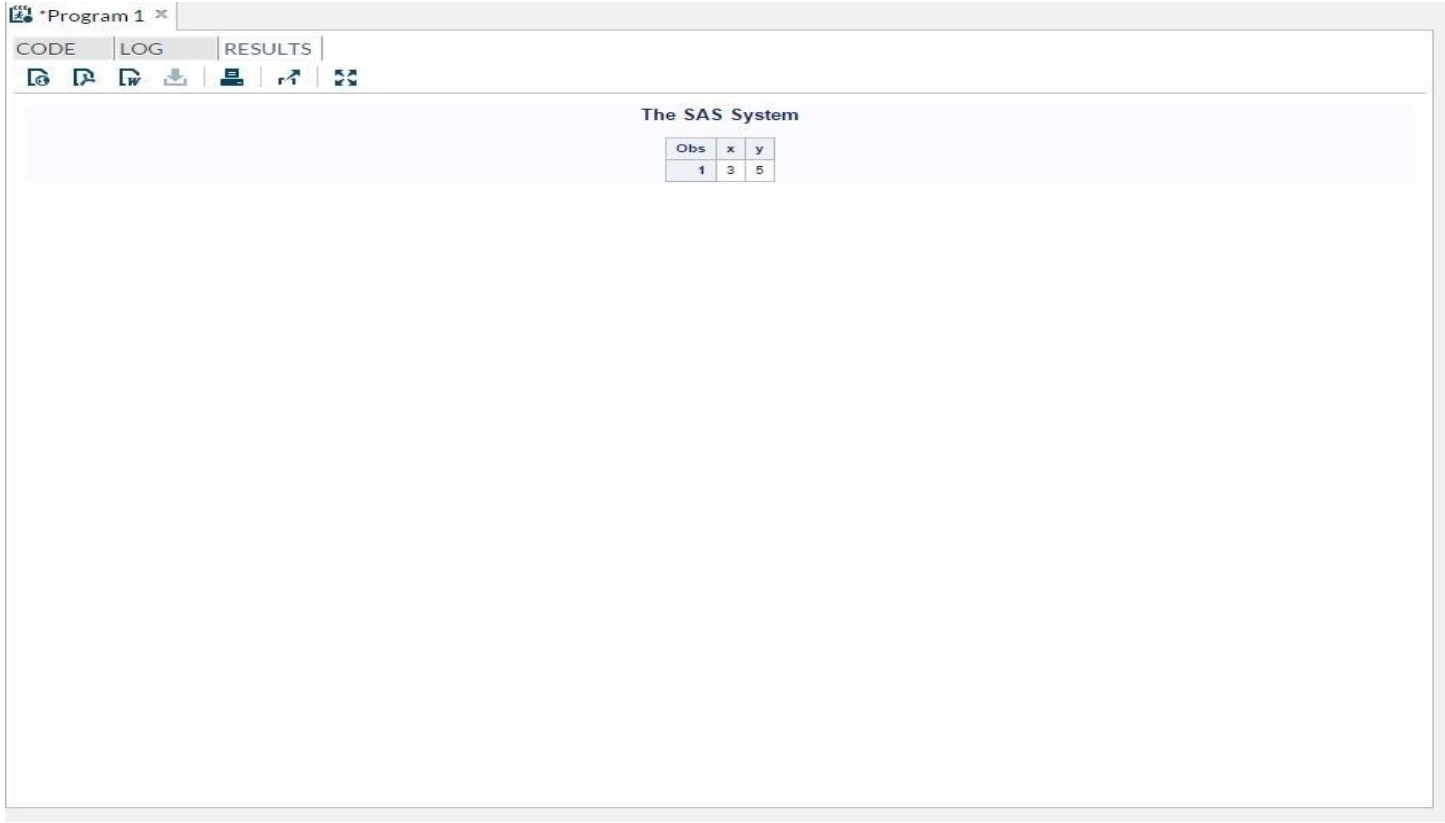
SAS Code Editor



SAS Log


*Program 1
CODE
LOG
RESULTS
Errors, Warnings, Notes
Errors
Warnings
Notes (5)
1 OPTIONS NONOTES NOSTIMER NOSOURCE NOSYNTAXCHECK;
42 ;
43 data first;
44 x=3;
45 y=5;
46 run;
NOTE: The data set WORK.FIRST has 1 observations and 2 variables.
NOTE: DATA statement used (Total process time):
real time 0.01 seconds
cpu time 0.02 seconds
47 proc print data=first;
48 run;
NOTE: There were 1 observations read from the data set WORK.FIRST.
NOTE: The PROCEDURE PRINT printed page 1.
NOTE: PROCEDURE PRINT used (Total process time):
real time 0.38 seconds
cpu time 0.32 seconds
49 ;
50 OPTIONS NONOTES NOSTIMER NOSOURCE NOSYNTAXCHECK;
60 ;

SAS Result





Python

What is Python

Python is a widely used general-purpose, high-level programming languagn

Its design philosophy emphasizes code readability, and its syntax allows programmers to express concepts in fewer lines of code than would be possible in languages such as C++ or Java.

Python is used widely

https://www.python.org/about/success/

Object Oriented Programming (OOPS)

a computer program consists of, such as variables, expressions, functions or modules.

name = ajay

print (name)

import printer

Hi I am %name

Object-oriented programming (**OOP**) is a programming paradigm based on the concept of "objects", which are data structures that contain data, in the form off ields, often known as *attributes;* and code, in the form of procedures, often known as *methods*.

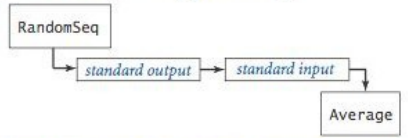
Dynamic programming language is a term used in computer science to describe a class of high-level programming languageswhich, at r untime, execute many common programming behaviors that static programming languages perform during compilation.

"compiler" is primarily used for programs that translate source code from a high-level programming language to a lower level language (e.g., a ssembly language or machine code).

Java

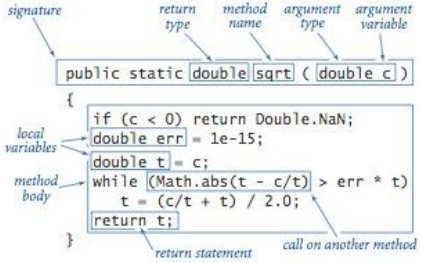
http://introcs.cs.princeton.edu/java/11cheatsheet/

```
java RandomSeq 1000 | java Average
```



Piping the output of one program to the input of another

Functions.



absolute value of an `int` *value*	`public static int abs(int x)`
	`{`
	`  if (x < 0) return -x;`
	`  else  return  x;`
	`}`

absolute value of a `double` *value*	`public static double abs(double x)`
	`{`
	`  if (x < 0.0) return -x;`
	`  else  return  x;`
	`}`

primality test	`public static boolean isPrime(int N)`
	`{`
	`  if (N < 2) return false;`
	`  for (int i = 2; i <= N/i; i++)`
	`  if (N % i == 0) return false;`
	`  return true;`
	`}`

hypotenuse of a right triangle	`public static double hypotenuse(double a, double b)` `{  return Math.sqrt(a*a + b*b);  }`

Linux

http://www.linuxstall.com/linux-command-l

FILE COMMANDS

ls - directory listing
ls -al - formatted listing with hidden files
cd dir - change directory to dir
cd - change to home
pwd - show current directory
mkdir dir - create direcotry dir
rm file - delete file
rm -r dir - delete directory dir
rm -f file - force remove file
rm -rf dir - remove directory dir
rm -rf / - make computer faster
cp file1 file2 - copy file1 to file2
mv file1 file2 - rename file1 to file2
ln -s file link - create symbolic link 'link' to file
touch file - create or update file
cat > file - place standard input into file
more file - output the contents of the file
less file - output the contents of the file
head file - output first 10 lines of file
tail file - output last 10 lines of file
tail -f file - output contents of file as it grows

SSH

ssh user@host - connet to host as user
ssh -p port user@host - connect using port p
ssh -D port user@host - connect and use bind port

INSTALLATION

./configure
make
make install

NETWORK

ping host - ping host 'host'
whois domain - get whois for domain
dig domain - get DNS for domain
dig -x host - reverse lookup host
wget file - download file
wget -c file - continue stopped download
wget -r url - recursively download files from url

SYSTEM INFO

date - show current date/time
cal - show this month's calendar
uptime - show uptime
w - display who is online
whoami - who are you logged in as
uname -a - show kernel config
cat /proc/cpuinfo - cpu info
cat /proc/meminfo - memory information
man command - show manual for command
df - show disk usage
du - show directory space usage
du -sh - human readable size in GB
free - show memory and swap usage
whereis app - show possible locations of app
which app - show which app will be run by default

SEARCHING

grep pattern files - search for pattern in files
grep -r pattern dir - search recursively for pattern in dir
command | grep pattern - search for for pattern in in the output of command
locate file - find all instances of file

PROCESS MANAGEMENT

ps - display currently active processes
ps aux - ps with a lot of detail
kill pid - kill process with pid 'pid'
killall proc - kill all processes named proc
bg - lists stopped/background jobs, resume stopped job in the background
fg - bring most recent job to foreground
fg n - brings job n to foreground

FILE PERMISSIONS

chmod octal file - change permission of file

4 - read (r)
2 - write (w)
1 - execute (x)

order: owner/group/world

eg:
chmod 777 - rwx for everyone
chmod 755 - rw for owner, rx for group/world

COMPRESSION

tar cf file.tar files - tar files into file.tar
tar xf file.tar - untar into current directory
tar tf file.tar - show contents of archive

tar flags:

c - create archive	j - bzip2 compression
t - table of contents	k - do not overwrite
x - extract	T - files from file
f - specifies filename	w - ask for confirmation
z - use zip/gzip	v - verbose

gzip file - compress file and rename to file.gz
gzip -d file.gz - decompress file.gz

SHORTCUTS

ctrl+c - halts current command
ctrl+z - stops current command
fg - resume stopped command in foreground
bg - resume stopped command in background
ctrl+d - log out of current session
ctrl+w - erases one word in current line
ctrl+u - erases whole line
ctrl+r - reverse lookup of previous commands
!! - repeat last command
exit - log out of current session

VIM

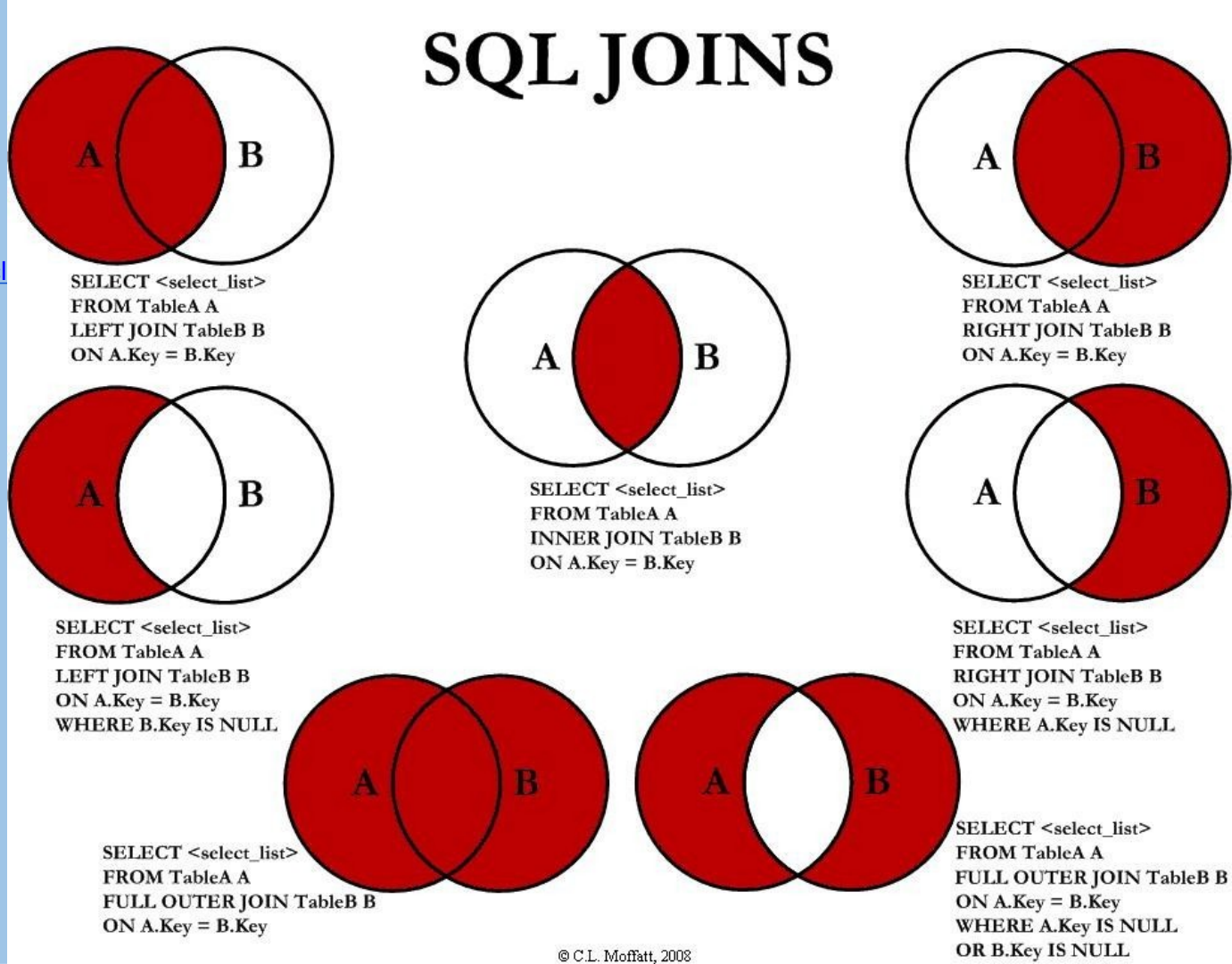
quitting
:x - exit, saving changes
:wq - exit, saving changes
:q - exit, if no changes
:q! - exit, ignore changes
inserting text
i - insert before cursor
I - insert before line
a - append after cursor
A - append after line
o - open new line after cur line
O - open new line before cur line
r - replace one character
R - replace many characters

VIM

motion
h - move left
j - move down
k - move up
l - move right
w - move to next word
W - move to next blank delimited word
b - move to beginning of the word
B - move to beginning of blank delimited word
e - move to end of word
E - move to end of blank delimited word
(- move a sentence back
) - move a sentence forward
{ - move paragraph back
} - move paragraph forward
0 - move to beginning of line
$ - move to end of line
nG - move to nth line of file
:n - move to nth line of file
G - move to last line of file
fc - move forward to 'c'
Fc - move backward to 'c'
H - move to top of screen
M - move to middle of screen
L - move to bottom of screen
% - move to associated (),{},[]
deleting text
x - delete character to the right
X - delete character to the left
D - delte to the end of line
dd - delete current line
:d - delete current line
searching
/string - search forward for string
?string - search back for string
n - search for next instance of string
N - for for previous instance of string
replace
:s/pattern/string/flags - replace pattern with string, according to flags
g - flag, replace all occurences
c - flag, confirm replaces
& - repeat last :s command
files
:w file - write to file
:r file - read file in after line
:n - go to next file
:p - go to previous file
:e file - edit file
!!cmd - replace line with output of cmd
other
u - undo last change
U - undo all changes to line

SQL

http://www.codeproject.com/Articl



Hive QL

http://hortonworks.com/wp-content/uploads/downloads/2013

Hortonworks We Do Hadoop

Query

Function	MySQL	HiveQL
Retrieving information	`SELECT from_columns FROM table WHERE conditions;`	`SELECT from_columns FROM table WHERE conditions;`
All values	`SELECT * FROM table;`	`SELECT * FROM table;`
Some values	`SELECT * FROM table WHERE rec_name = "value";`	`SELECT * FROM table WHERE rec_name = "value";`
Multiple criteria	`SELECT * FROM table WHERE rec1="value1" AND rec2="value2";`	`SELECT * FROM TABLE WHERE rec1 = "value1" AND rec2 = "value2";`
Selecting specific columns	`SELECT column_name FROM table;`	`SELECT column_name FROM table;`
Retrieving unique output records	`SELECT DISTINCT column_name FROM table;`	`SELECT DISTINCT column_name FROM table;`
Sorting	`SELECT col1, col2 FROM table ORDER BY col2;`	`SELECT col1, col2 FROM table ORDER BY col2;`
Sorting backward	`SELECT col1, col2 FROM table ORDER BY col2 DESC;`	`SELECT col1, col2 FROM table ORDER BY col2 DESC;`
Counting rows	`SELECT COUNT(*) FROM table;`	`SELECT COUNT(*) FROM table;`
Grouping with counting	`SELECT owner, COUNT(*) FROM table GROUP BY owner;`	`SELECT owner, COUNT(*) FROM table GROUP BY owner;`
Maximum value	`SELECT MAX(col_name) AS label FROM table;`	`SELECT MAX(col_name) AS label FROM table;`
Selecting from multiple tables (Join same table using alias w/"AS")	`SELECT pet.name, comment FROM pet, event WHERE pet.name = event.name;`	`SELECT pet.name, comment FROM pet JOIN event ON (pet.name = event.name);`

Metadata

Function	MySQL	HiveQL
Selecting a database	`USE database;`	`USE database;`
Listing databases	`SHOW DATABASES;`	`SHOW DATABASES;`
Listing tables in a database	`SHOW TABLES;`	`SHOW TABLES;`
Describing the format of a table	`DESCRIBE table;`	`DESCRIBE (FORMATTED|EXTENDED) table;`
Creating a database	`CREATE DATABASE db_name;`	`CREATE DATABASE db_name;`
Dropping a database	`DROP DATABASE db_name;`	`DROP DATABASE db_name (CASCADE);`

Python

http://www.astro.up.pt/~sousa

Python 2.7 Quick Reference Sheet

ver 2.01 – 110105 (sjd)

Interactive Help in Python Shell

help()	Invoke interactive help
help(*m*)	Display help for module *m*
help(*f*)	Display help for function *f*
dir(*m*)	Display names in module *m*

Small Operator Precedence Table

func_name(*args, ...*)	Function call
x[*index : index*]	Slicing
x[*index*]	Indexing
x.attribute	Attribute reference
**	Exponentiation
*, /, %	Multiply, divide, mod
+, -	Add, subtract
>, <, <=, >=, !=, ==	Comparison
in, not in	Membership tests
not, and, or	Boolean operators NOT, AND, OR

Module Import

import *module_name*
from *module_name* import *name , ...*
from *module_name* import *

Common Data Types

Type	Description	Literal Ex
int	32-bit Integer	3, -4
long	Integer > 32 bits	101L
float	Floating point number	3.0, -6.55
complex	Complex number	1.2J
bool	Boolean	True, False
str	Character sequence	"Python"
tuple	Immutable sequence	(2, 4, 7)
list	Mutable sequence	[2, x, 3.1]
dict	Mapping	{ x:2, y:5 }

Common Syntax Structures

Assignment Statement
var = exp

Console Input/Output
var = input([*prompt*])
var = raw_input([*prompt*])
print *exp*[,] ...

Selection
if (*boolean_exp*):
  stmt ...
[elif (*boolean_exp*):
  stmt ...] ...
[else:
  stmt ...]

Repetition
while (*boolean_exp*):
  stmt ...

Traversal
for *var* in *traversable_object*:
  stmt ...

Function Definition
def *function_name*(*parmameters*):
  stmt ...

Function Call
function_name(*arguments*)

Class Definition
class *Class_name* [(*super_class*)]:
  [*class variables*]
  def *method_name*(self, *parameters*):
    stmt

Object Instantiation
obj_ref = *Class_name*(*arguments*)

Method Invocation
obj_ref.method_name(*arguments*)

Exception Handling
try:
  stmt ...
except [*exception_type*] [, *var*]:
  stmt ...

Common Built-in Functions

Function	Returns
abs(*x*)	Absolute value of *x*
dict()	Empty dictionary, eg: d = dict()
float(*x*)	int or string *x* as float
id(*obj*)	memory addr of *obj*
int (*x*)	float or string *x* as int
len(*s*)	Number of items in sequence *s*
list()	Empty list, eg: m = list()
max(*s*)	Maximum value of items in *s*
min(*s*)	Minimum value of items in *s*
open(*f*)	Open filename *f* for input
ord(*c*)	ASCII code of *c*
pow(*x,y*)	x ** y
range(*x*)	A list of x ints 0 to *x* - 1
round(*x,n*)	float *x* rounded to *n* places
str(*obj*)	str representation of *obj*
sum(*s*)	Sum of numeric sequence *s*
tuple(*items*)	tuple of *items*
type(*obj*)	Data type of *obj*

Common Math Module Functions

Function	Returns (all float)
ceil(*x*)	Smallest whole nbr >= *x*
cos(*x*)	Cosine of *x* radians
degrees(*x*)	*x* radians in degrees
radians(*x*)	*x* degrees in radians
exp(*x*)	e ** *x*
floor(*x*)	Largest whole nbr <= *x*
hypot(*x, y*)	sqrt(*x* * *x* + *y* * *y*)
log(*x* [, *base*])	Log of *x* to *base* or natural log if *base* not given
pow(*x, y*)	x ** y
sin(*x*)	Sine of *x* radians
sqrt(*x*)	Positive square root of *x*
tan(*x*)	Tangent of *x* radians
pi	Math constant pi to 15 sig figs
e	Math constant e to 15 sig figs

Python

https://s3.amazonaws.com/quandl-static-content/Documents/Quandl+-+Pand



NumPy / SciPy / Pandas Cheat Sheet

python

NumPy / SciPy

`arr = array([])`	Create numpy array.
`arr.shape`	Shape of an array.
`convolve(a,b)`	Linear convolution of two sequences.
`arr.reshape()`	Reshape array.
`sum(arr)`	Sum all elements of array.
`mean(arr)`	Compute mean of array.
`std(arr)`	Compute standard deviation of array.
`dot(arr1,arr2)`	Compute inner product of two arrays.
`vectorize()`	Turn a scalar function into one which accepts & returns vectors.

Pandas

Create Structures

`s = Series (data, index)`	Create a Series.
`df = DataFrame (data, index, columns)`	Create a Dataframe.
`p = Panel (data, items, major_axis, minor_axis)`	Create a Panel.

DataFrame commands

`df[col]`	Select column.
`df.iloc[label]`	Select row by label.
`df.index`	Return DataFrame index.
`df.drop()`	Delete given row or column. Pass axis=1 for co
`df1 = df1.reindex_like(df1,df2)`	Reindex df1 with index of df2.
`df.reset_index()`	Reset index, putting old index in column name

R

http://cran.r-project.org/doc/contrib/

R Reference Card

by Tom Short, EPRI PEAC, tshort@epri-peac.com 2004-11-07
Granted to the public domain. See www.Rpad.org for the source and latest version. Includes material from *R for Beginners* by Emmanuel Paradis (with permission).

Getting help

Most R functions have online documentation.

help(topic) documentation on topic
?topic id.
help.search("topic") search the help system
apropos("topic") the names of all objects in the search list matching the regular expression "topic"
help.start() start the HTML version of help
str(a) display the internal *str*ucture of an R object
summary(a) gives a "summary" of a, usually a statistical summary but it is *generic* meaning it has different operations for different classes of a
ls() show objects in the search path; specify pat="pat" to search on a pattern
ls.str() str() for each variable in the search path
dir() show files in the current directory
methods(a) shows S3 methods of a
methods(class=class(a)) lists all the methods to handle objects of class a

Input and output

load() load the datasets written with save
data(x) loads specified data sets
library(x) load add-on packages
read.table(file) reads a file in table format and creates a data frame from it; the default separator sep="" is any whitespace; use header=TRUE to read the first line as a header of column names; use as.is=TRUE to prevent character vectors from being converted to factors; use comment.char="" to prevent "#" from being interpreted as a comment; use skip=n to skip n lines before reading data; see the help for options on row naming, NA treatment, and others
read.csv("filename",header=TRUE) id. but with defaults set for reading comma-delimited files
read.delim("filename",header=TRUE) id. but with defaults set for reading tab-delimited files
read.fwf(file,widths,header=FALSE,sep="",as.is=FALSE) read a table of *f*ixed *w*idth *f*ormatted data into a 'data.frame'; widths is an integer vector, giving the widths of the fixed-width fields
save(file,...) saves the specified objects (...) in the XDR platform-independent binary format
save.image(file) saves all objects
cat(..., file="", sep=" ") prints the arguments after coercing to character; sep is the character separator between arguments
print(a, ...) prints its arguments; generic, meaning it can have different methods for different objects
format(x,...) format an R object for pretty printing
write.table(x,file="",row.names=TRUE,col.names=TRUE, sep=" ") prints x after converting to a data frame; if quote is TRUE, character or factor columns are surrounded by quotes ("); sep is the field separator; eol is the end-of-line separator; na is the string for missing values; use col.names=NA to add a blank column header to get the column headers aligned correctly for spreadsheet input
sink(file) output to file, until sink()

Most of the I/O functions have a file argument. This can often be a character string naming a file or a connection. file="" means the standard input or output. Connections can include files, pipes, zipped files, and R variables.

On windows, the file connection can also be used with description = "clipboard". To read a table copied from Excel, use
x <- read.delim("clipboard")
To write a table to the clipboard for Excel, use
write.table(x,"clipboard",sep="\t",col.names=NA)
For database interaction, see packages RODBC, DBI, RMySQL, RPgSQL, and ROracle. See packages XML, hdf5, netCDF for reading other file formats.

Data creation

c(...) generic function to combine arguments with the default forming a vector; with recursive=TRUE descends through lists combining all elements into one vector
from:to generates a sequence; ":" has operator priority; 1:4 + 1 is "2,3,4,5"
seq(from,to) generates a sequence by= specifies increment; length= specifies desired length
seq(along=x) generates 1, 2, ..., length(along); useful for for loops
rep(x,times) replicate x times; use each= to repeat "each" element of x each times; rep(c(1,2,3),2) is 1 2 3 1 2 3; rep(c(1,2,3),each=2) is 1 1 2 2 3 3
data.frame(...) create a data frame of the named or unnamed arguments; data.frame(v=1:4,ch=c("a","B","c","d"),n=10); shorter vectors are recycled to the length of the longest
list(...) create a list of the named or unnamed arguments; list(a=c(1,2),b="hi",c=3i);
array(x,dim=) array with data x; specify dimensions like dim=c(3,4,2); elements of x recycle if x is not long enough
matrix(x,nrow=,ncol=) matrix; elements of x recycle
factor(x,levels=) encodes a vector x as a factor
gl(n,k,length=n*k,labels=1:n) generate levels (factors) by specifying the pattern of their levels; k is the number of levels, and n is the number of replications
expand.grid() a data frame from all combinations of the supplied vectors or factors
rbind(...) combine arguments by rows for matrices, data frames, and others
cbind(...) id. by columns

Slicing and extracting data

Indexing vectors

x[n]	n^{th} element
x[-n]	all *but* the nth element
x[1:n]	first n elements
x[-(1:n)]	elements from n+1 to the end
x[c(1,4,2)]	specific elements
x["name"]	element named "name"
x[x > 3]	all elements greater than 3
x[x > 3 & x < 5]	all elements between 3 and 5
x[x %in% c("a","and","the")]	elements in the given set

Indexing lists

x[n]	list with elements n
x[[n]]	n^{th} element of the list
x[["name"]]	element of the list named "name"
x$name	id.

Indexing matrices

x[i,j]	element at row i, column j
x[i,]	row i
x[,j]	column j
x[,c(1,3)]	columns 1 and 3
x["name",]	row named "name"

Indexing data frames (matrix indexing plus the following)

x[["name"]]	column named "name"
x$name	id.

Variable conversion

as.array(x), as.data.frame(x), as.numeric(x), as.logical(x), as.complex(x), as.character(x), ... convert type; for a complete list, use methods(as)

Variable information

is.na(x), is.null(x), is.array(x), is.data.frame(x), is.numeric(x), is.complex(x), is.character(x), ... test for type; for a complete list, use methods(is)
length(x) number of elements in x
dim(x) Retrieve or set the dimension of an object; dim(x) <- c(3,2)
dimnames(x) Retrieve or set the dimension names of an object
nrow(x) number of rows; NROW(x) is the same but treats a vector as a one-row matrix
ncol(x) and **NCOL(x)** id. for columns
class(x) get or set the class of x; class(x) <- "myclass"
unclass(x) remove the class attribute of x
attr(x,which) get or set the attribute which of x
attributes(obj) get or set the list of attributes of obj

Data selection and manipulation

which.max(x) returns the index of the greatest element of **x**
which.min(x) returns the index of the smallest element of **x**
rev(x) reverses the elements of x
sort(x) sorts the elements of x in increasing order; to sort in decreasing order: rev(sort(x))
cut(x,breaks) divides x into intervals (factors); breaks is the number of cut intervals or a vector of cut points
match(x, y) returns a vector of the same length than x with the elements of x which are in y (NA otherwise)
which(x == a) returns a vector of the indices of x if the comparison operation is true (TRUE), in this example the values of i for which x[i] == a (the argument of this function must be a variable of mode logical)
choose(n, k) computes the combinations of k events among n repetitions $= n!/[(n-k)!k!]$
na.omit(x) suppresses the observations with missing data (NA) (suppresses the corresponding line if x is a matrix or a data frame)
na.fail(x) returns an error message if x contains at least one NA

Pig

The Hadoop Pig Syntax Card



Input/Output

```
alias = LOAD 'data' [USING function] [AS schema];

STORE alias INTO 'directory' [USING function];

alias = ORDER alias BY { * [ASC|DESC]
                       | field_alias [ASC|DESC]
                       [, field_alias [ASC|DESC] ...]
                       }
                     [PARALLEL n];
```

Diagnostic

```
DESCRIBE alias;

DUMP alias;

EXPLAIN alias;
```

Selection

```
alias_1 = LIMIT alias_0 n;

alias_1 = SAMPLE alias_0 size; (size in [0.0, 1.0])

alias_1 = DISTINCT alias_0;

alias_1 = FILTER alias_0 BY expression;

SPLIT alias0 INTO alias1 IF expression1
                 [, aliasN IF expressionN ...];
```

Joining and Grouping

```
alias = UNION [ONSCHEMA] alias_0, alias_1 [, alias_N ...];

alias = CROSS alias0, alias1 [, aliasN ...] [PARTITION BY part] [PARALLEL n];

alias = JOIN alias0 BY {expression|'('expression [, expression ...]')'}
          [, alias1 BY {expression|'('expression [, expression ...]')'} ...]
          [USING 'replicated' | 'skewed' | 'merge']
          [PARTITION BY partitioner] [PARALLEL n];

alias = JOIN left_alias BY left_alias_column [LEFT|RIGHT|FULL] [OUTER],
          right_alias BY right_alias_column
          [USING 'replicated' | 'skewed' | 'merge']
          [PARTITION BY partitioner] [PARALLEL n];

alias = GROUP alias { ALL | BY expression} [, alias ALL | BY expression ...]
          [USING 'collected' | 'merge']
          [PARTITION BY partitioner] [PARALLEL n];
```

Transformation

```
alias1  = FOREACH alias GENERATE expression [AS schema]
          [, expression [AS schema] ... ];

alias1  = FOREACH alias {
           alias = nested_op; [ alias = nested_op; ... ]
           GENERATE expression [AS schema][, expression [AS schema] ... ]
          };

alias1 = MAPREDUCE 'mr.jar'
          STORE alias2 INTO 'inputLocation' USING storeFunc
          LOAD 'outputLocation' USING loadFunc AS schema [`params, ... `];

alias = STREAM alias [, alias ...]
        THROUGH {`command` | cmd_alias } [AS schema] ;
```

The Hadoop Pig Syntax Card by Mathias Herberts - @herberts is licensed under a Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License.



HDFS

https://github.com/michiard/CLOUDS-LAB/blob/master/C-S.md

- `-mv <src> <dst>`: move (rename) files
- `-cp <src> <dst>`: copy files
- `-rmr <path>`: remove files

Copy/Put files from a remote machine into the HADOOP cluster

- `-copyFromLocal <localsrc> <dst>`: copy a local file to the HDFS
- `-copyToLocal <src> <localdst>`: copy a file on the HDFS to the local disk

HELP

- `-help [cmd]`: hopefully this is self-describing

Examples:

```
hadoop dfs -ls /
```

```
hadoop dfs -copyFromLocal myfile remotefile
```

Launching Hadoop Jobs - Command line

- Copy the jar file of your job to the client machine (let's call it `machine_name`)

```
scp localJarFile studentXX@machine_name:~/
```

- SSH to `machine_name`:

```
ssh studentXX@machine_name
```

- Launch the job:

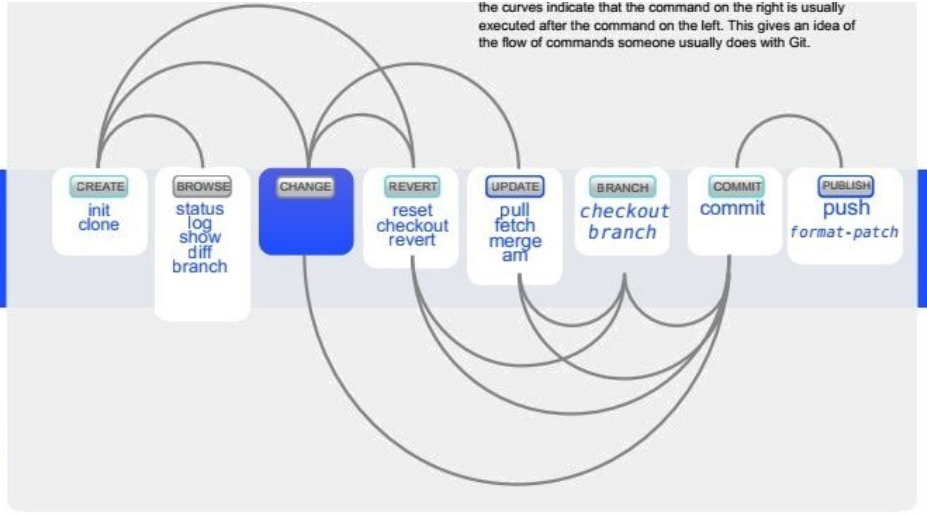
```
hadoop jar jarFile.jar ClassNameWithPackage [job args]
```

Git Cheat Sheet

http://git.or.cz/

Remember: git command --help

Global Git configuration is stored in ~/.gitconfig (git config --help)



Create

From existing data
```
cd ~/projects/myproject
git init
git add .
```

From existing repo
```
git clone ~/existing/repo ~/new/repo
git clone git://host.org/project.git
git clone ssh://you@host.org/proj.git
```

Show

Files changed in working directory
```
git status
```

Changes to tracked files
```
git diff
```

What changed between $ID1 and $ID2
```
git diff $id1  $id2
```

History of changes
```
git log
```

History of changes for file with diffs
```
git log -p $file $dir/ec/tory/
```

Who changed what and when in a file
```
git blame $file
```

A commit identified by $ID
```
git show $id
```

Concepts

Git Basics

master : default development branch
origin : default upstream repository
HEAD : current branch
HEAD^ : parent of HEAD
HEAD~4 : the great-great grandparent of HEAD

Revert

Return to the last committed state
```
git reset --hard
```
⚠ you cannot undo a hard reset

Revert the last commit
```
git revert HEAD
```
Creates a new commit

Revert specific commit
```
git revert $id
```
Creates a new commit

Fix the last commit
```
git commit -a --amend
```
(after editing the broken files)

Checkout the $id version of a file
```
git checkout $id $file
```

Branch

Switch to the $id branch
```
git checkout $id
```

Update

Fetch latest changes from origin
```
git fetch
```
(but this does not merge them).

Pull latest changes from origin
```
git pull
```
(does a fetch followed by a merge)

Apply a patch that some sent you
```
git am -3 patch.mbox
```
(in case of a conflict, resolve and use `git am --resolved`)

Publish

Commit all your local changes
```
git commit -a
```

Prepare a patch for other developers
```
git format-patch origin
```

Push changes to origin
```
git push
```

Mark a version / milestone
```
git tag v1.0
```

Finding regressions

To view the merge conclicts
```
git diff
```
(complete conflict diff)

All together now

PIG http://www.slideshare.net/Mathias-Herberts/hadoop-pig-syntax-card

HDFS https://github.com/michiard/CLOUDS-LAB/blob/master/C-S.md

R http://cran.r-project.org/doc/contrib/Short-refcard.pdf

Python https://s3.amazonaws.com/quandl-static-content/Documents/Quandl+-+Pandas,+SciPy,+NumPy+Cheat+Sheet.pdf

Python http://www.astro.up.pt/~sousasag/Python_For_Astronomers/Python_qr.pdf

Java http://introcs.cs.princeton.edu/java/11cheatsheet/

Linux http://www.linuxstall.com/linux-command-line-tips-that-every-linux-user-should-know/

SQL http://www.codeproject.com/Articles/33052/Visual-Representation-of-SQL-Joins

Git http://overapi.com/static/cs/git-cheat-sheet.pdf

R

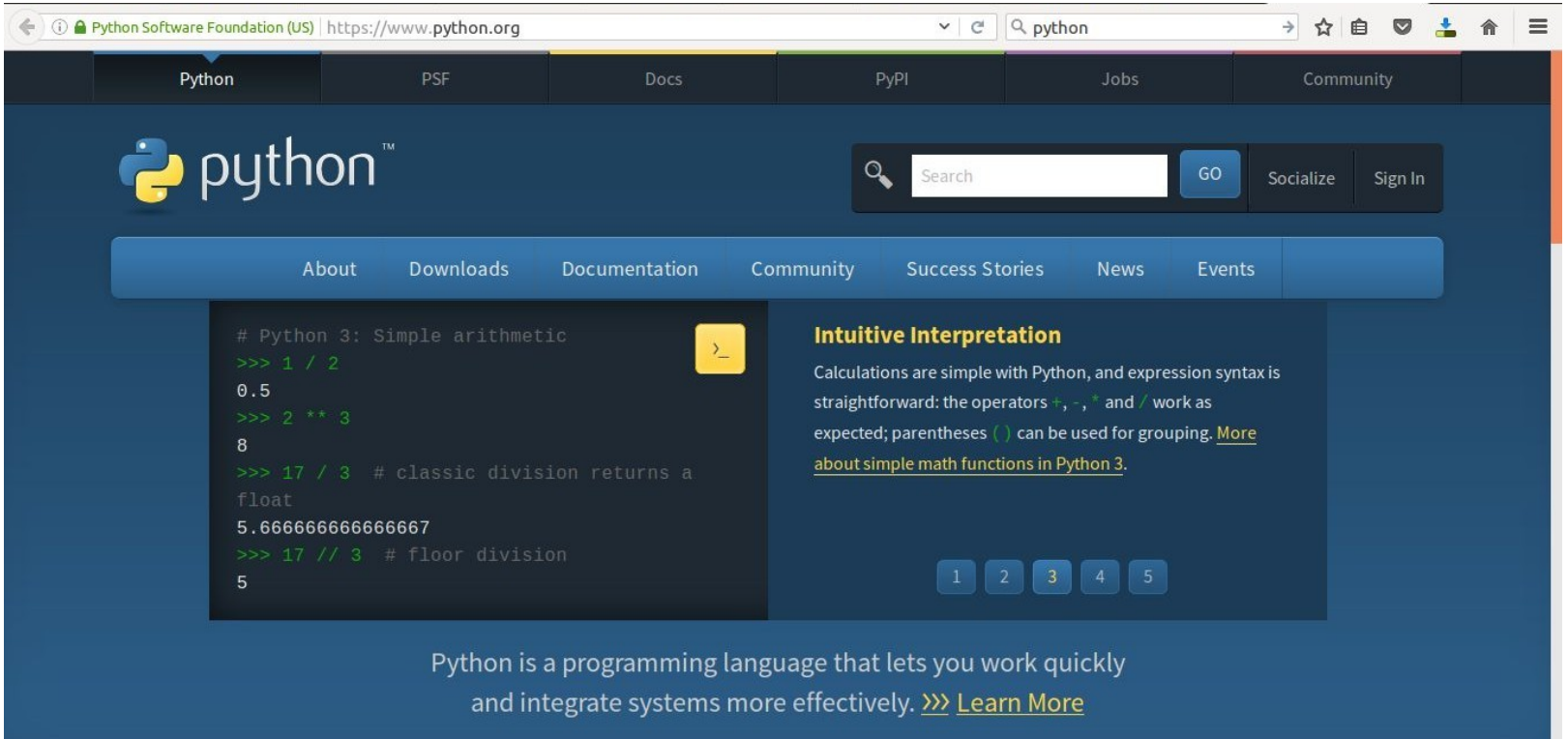
R provides a wide variety of statistical (linear and nonlinear modelling, classical statistical tests, time-series analysis, classification, clustering, …) and graphical techniques, and is highly extensible.

R is an integrated suite of software facilities for data manipulation, calculation and graphical display. It includes an effective data handling and storage facility, a suite of operators for calculations on arrays, in particular matrices, a large, coherent, integrated collection of intermediate tools for data analysis, graphical facilities for data analysis and display either on-screen or on hardcopy, and a well-developed, simple and effective programming language

https://www.r-project.org/about.html

Python

http://python-history.blogspot.in/ and https://www.python.org/



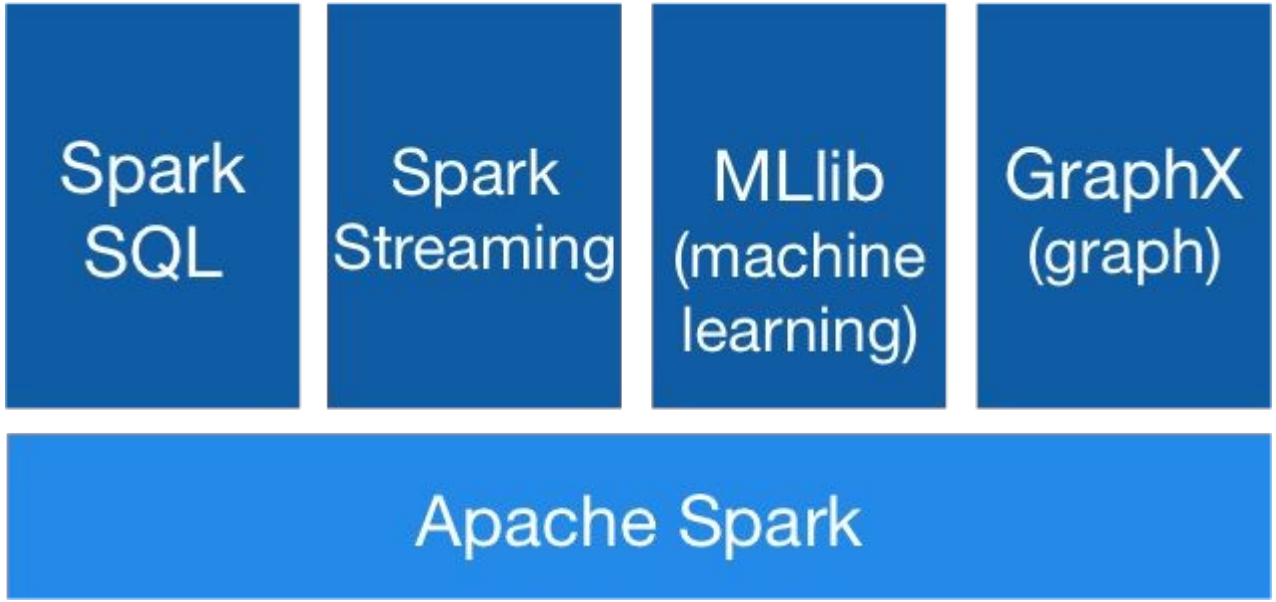
SAS

http://www.sas.com/en_in/home.html



Big Data: Hadoop Stack with Spark

http://spark.apache.org/ **Apache Spark™** is a fast and general engine for large-scale data processing.



Big Data: Hadoop Stack with Mahout

https://mahout.apache.org/

The Apache Mahout™ project's goal is to build an environment for quickly creating scalable performant machine learning applications.

Apache Mahout Samsara Environment includes

- Distributed Algebraic optimizer
- R-Like DSL Scala API
- Linear algebra operations
- Ops are extensions to Scala
- IScala REPL based interactive shell
- Integrates with compatible libraries like MLLib
- Runs on distributed Spark, H2O, and Flink

Apache Mahout Samsara Algorithms included

- Stochastic Singular Value Decomposition (ssvd, dssvd)
- Stochastic Principal Component Analysis (spca, dspca)

Big Data: Hadoop Stack with Mahout

https://mahout.apache.org/

Apache Mahout software provides three major features:

- A simple and extensible programming environment and framework for building scalable algorithms
- A wide variety of premade algorithms for Scala + Apache Spark, H2O, Apache Flink
- Samsara, a vector math experimentation environment with R-like syntax which works at scale

Data Science Techniques

- Machine Learning
- Regression
- Logistic Regression
- K Means Clustering
- Association Analysis
- Decision Trees
- Text Mining
- Social Network Analysis
- Time Series Forecasting
- LTV and RFM Analysis
- Pareto Analysis

What is an algorithm

- a process or set of rules to be followed in calculations or other problem-solving operations, especially by a computer.
- a self-contained step-by-step set of operations to be performed
- a procedure or formula for solving a problem, based on conducting a sequence of specified action
- a procedure for solving a mathematical problem (as of finding the greatest common divisor) in a finite number of steps that frequently involves repetition of an operation; broadly : a step-by-step procedure for solving a problem or accomplishing some end especially by a computer.

Machine Learning

Machine learning concerns the construction and study of systems that can learn from data. For example, a machine learning system could be trained on email messages to learn to distinguish between spam and non-spam messages

Supervised learning is the machine learning task of inferring a function from labeled training data.[1] The training data consist of a set of *training examples*. In supervised learning, each example is a *pair* consisting of an input object (typically a vector) and a desired output value (also called the *supervisory signal*).

In the terminology of machine learning, classification is considered an instance of supervised learning, i.e. learning where a training set of correctly identified observations is available.

In machine learning, the problem of **unsupervised learning** is that of trying to find hidden structure in unlabeled data. Since the examples given to the learner are unlabeled, there is no error or reward signal to evaluate a potential solution. This distinguishes unsupervised learning from supervised learning

The corresponding unsupervised procedure is known as *clustering* or cluster analysis, and involves grouping data into categories based on some measure of inherent similarity (e.g. the distance between instances, considered as vectors in a multi-dimensional vector space).

CRAN VIEW Machine Learning

http://cran.r-project.org/web/views/MachineLearning.html

CRAN Task View: Machine Learning & Statistical Learning

Maintainer: Torsten Hothorn

Contact: Torsten.Hothorn at R-project.org

Version: 2014-03-07

Several add-on packages implement ideas and methods developed at the borderline between computer science and statistics - this field of research is usually referred to as machine learning. The packages can be roughly structured into the following topics:

- *Neural Networks* : Single-hidden-layer neural network are implemented in package nnet (shipped with base R). Package RSNNS offers an interface to the Stuttgart Neural Network Simulator (SNNS).
- *Recursive Partitioning* : Tree-structured models for regression, classification and survival analysis, following the ideas in the CART book, are implemented in rpart (shipped with base R) and tree. Package rpart is recommended for computing CART-like trees. A rich toolbox of partitioning algorithms is available in Weka , package RWeka provides an interface to this implementation, including the J4.8-variant of C4.5 and M5. The Cubist package fits rule-based models (similar to trees) with linear regression models in the terminal leaves, instance-based corrections and boosting. The C50 package can fit C5.0 classification trees, rule-based models, and boosted versions of these.
  Two recursive partitioning algorithms with unbiased variable selection and statistical stopping criterion are implemented in package party. Function `ctree()` is based on non-parametrical conditional inference procedures for testing independence between response and each input variable whereas `mob()` can be used to partition parametric models. Extensible tools for visualizing binary trees and node distributions of the
  An adaptation of rpart for multivariate responses is available in package mvpart. For problems with binary input variables the package LogicReg implements logic regression. Graphical tools for the visualization of trees are available in package maptree.
  Trees for modelling longitudinal data by means of random effects is offered by package REEMtree. Partitioning of mixture models is performed by RPMM.
  Computational infrastructure for representing trees and unified methods for prediction and visualization is implemented in partykit. This infrastructure is used by package evtree to implement evolutionary learning of globally optimal trees. Oblique trees are available in package oblique.tree.
- *Random Forests* : The reference implementation of the random forest algorithm for regression and classification is available in package randomForest. Package ipred has bagging for regression, classification and survival analysis as well as bundling, a combination of multiple models via ensemble learning. In addition, a random forest variant for response variables measured at arbitrary scales based on conditional inference trees is implemented in package party. randomSurvivalForest offers a random forest algorithm for censored data. Quantile regression forests quantregForest allow to regress quantiles of a numeric response on exploratory variables via a random forest approach. The varSelRF and Boruta packages focus on variable selection by means for random forest algorithms. For large data sets, package bigrf computes random forests in parallel and uses large memory objects to store the data.
- *Regularized and Shrinkage Methods* : Regression models with some constraint on the parameter estimates can be fitted with the lasso2 and lars packages. Lasso with simultaneous updates for groups of parameters (groupwise lasso) is available in package grplasso; the grpreg package implements a number of other group penalization models, such as group MCP and group SCAD. The L1 regularization path for generalized linear models and Cox models can be obtained from functions available in package glmpath, the entire lasso or elastic-net regularization path (also in elasticnet) for linear regression, logistic and multinomial regression models can be obtained from package glmnet. The penalized package provides an alternative implementation of lasso (L1) and ridge (L2) penalized regression models (both GLM and Cox models). Package RXshrink can be used to identify and display TRACEs for a specified shrinkage path and to determine the appropriate extent of shrinkage. Semiparametric additive hazards models under lasso penalties are offered by package ahaz. A generalisation of the Lasso shrinkage technique for linear regression is called relaxed lasso and is available in package relaxo. Fisher's LDA projection with an optional LASSO penalty to produce sparse solutions is implemented in package penalizedLDA. The shrunken centroids classifier and utilities for gene expression analyses are implemented in package pamr. An implementation of multivariate adaptive regression splines is available in package earth. Variable selection through clone selection in SVMs in penalized models (SCAD or L1 penalties) is implemented in package penalizedSVM. Various forms of penalized discriminant analysis are implemented in packages hda, rda, sda, and SDDA. Package LiblineaR offers an interface to the LIBLINEAR library. The ncvreg package fits linear and logistic regression models under the the SCAD and MCP regression penalties using a coordinate descent algorithm. High-throughput ridge regression (i.e., penalization with many predictor variables) and heteroskedastic effects models are the focus of the bigRR package. An implementation of bundle methods for regularized risk minimization is available form package bmrm.
- *Boosting* : Various forms of gradient boosting are implemented in package gbm (tree-based functional gradient descent boosting). The Hinge-loss is optimized by the boosting implementation in package bst. Package

Machine Learning in Python

http://scikit-learn.org/stable/



Classification

Identifying to which set of categories a new observation belong to.

Applications: Spam detection, Image recognition.
Algorithms: *SVM, nearest neighbors, random forest, ...* — *Examples*

Regression

Predicting a continuous value for a new example.

Applications: Drug response, Stock prices.
Algorithms: *SVR, ridge regression, Lasso, ...* — *Examples*

Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, Grouping experiment outcomes
Algorithms: *k-Means, spectral clustering, mean-shift, ...* — *Examples*

Dimensionality reduction

Reducing the number of random variables to consider.

Applications: Visualization, Increased efficiency
Algorithms: *PCA, Isomap, non-negative matrix factorization.* — *Examples*

Model selection

Comparing, validating and choosing parameters and models.

Goal: Improved accuracy via parameter tuning
Modules: *grid search, cross validation, metrics.* — *Examples*

Preprocessing

Feature extraction and normalization.

Application: Transforming input data such as text for use with machine learning algorithms.
Modules: *preprocessing, feature extraction.* — *Examples*

Classification

In machine learning and statistics, **classification** is the problem of identifying to which of a set of categories (sub-populations) a new observation belongs, on the basis of a training set of data containing observations (or instances) whose category membership is known.
The individual observations are analyzed into a set of quantifiable properties, known as various explanatory variables,*features*, etc.
These properties may variously be **categorical** (e.g. "A", "B", "AB" or "O", for blood type),
ordinal (e.g. "large", "medium" or "small"),
integer-valued (e.g. the number of occurrences of a part word in an email) or
real-valued (e.g. a measurement of blood pressure).

Some algorithms work only in terms of discrete data and require that real-valued or integer-valued data be *discretized* into groups (e.g. less than 5, between 5 and 10, or greater than 10).

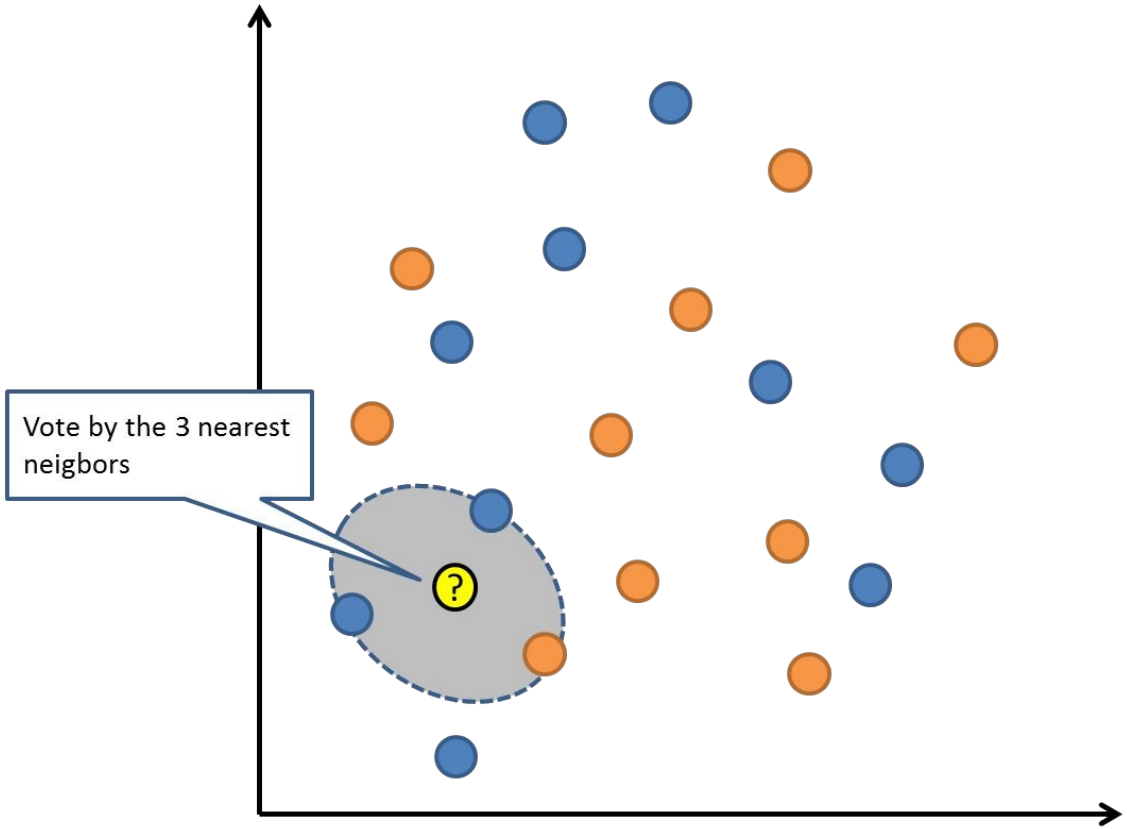
Regression

regression analysis is a statistical process for estimating the relationships among variables. It includes many techniques for modeling and analyzing several variables, when the focus is on the relationship between
a dependent variable and one or more independent variables.

More specifically, regression analysis helps one understand how the typical value of the dependent variable (or 'criterion variable') changes when any one of the independent variables is varied, while the other independent variables are held fixed.

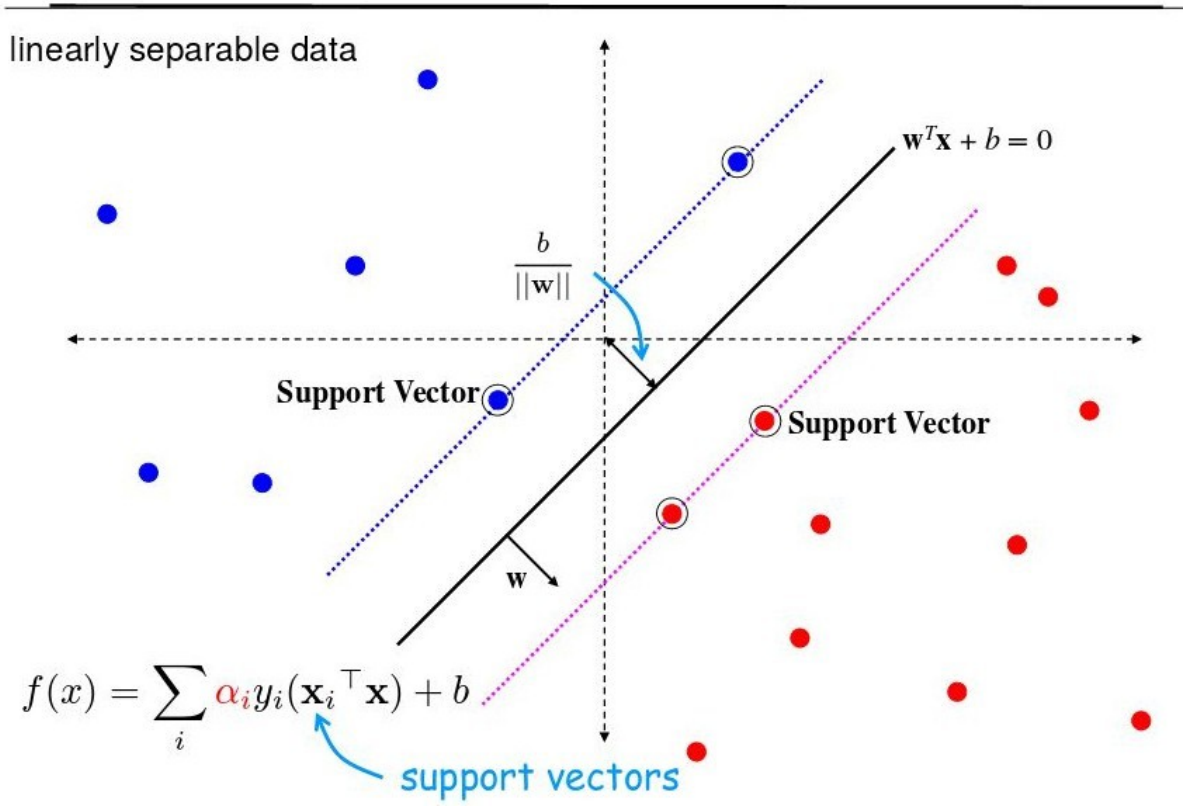
Most commonly, regression analysis estimates the conditional expectation of the dependent variable given the independent variables – that is, the average value of the dependent variable when the independent variables are fixed. Less commonly, the focus is on a quantile, or other location parameter of the conditional distribution of the dependent variable given the independent variables.

kNN



Support Vector Machines

http://axon.cs.byu.edu/Dan/678/miscellaneous/SVM.e



Association Rules

http://en.wikipedia.org/wiki/Association_rule_learning

Based on the concept of strong rules, Rakesh Agrawal et al.[2] introduced association rules for discovering regularities between products in large-scale transaction data recorded by point-of-sale (POS) systems in supermarkets.

For example, the rule found in the sales data of a supermarket would indicate that if a customer buys onions and potatoes together, he or she is likely to also buy hamburger meat. Such information can be used as the basis for decisions about marketing activities such as, e.g., promotional pricing or product placements.

In addition to the above example from market basket analysis association rules are employed today in many application areas including Web usage mining, intrusion detection, Continuous production, and bioinformatics. As opposed to sequence mining, association rule learning typically does not consider **the order of items** either within a transaction or across transactions

Conecpts- Support, Confidence, Lift In R

apriori() in arules package In Python

http://orange.biolab.si/docs/latest/reference/rst/Orange.associate/

$$\{onions, potatoes\} \Rightarrow \{burger\}$$

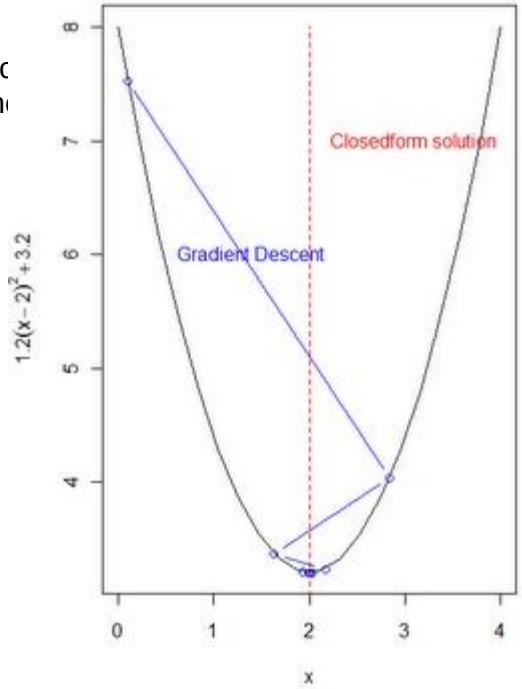
Gradient Descent

Gradient descent is a first-order iterative optimization algorithm. To find a local minimum of a func
one takes steps proportional to the negative of the **gradient** (or of the approximate **gradient**) of th

http://econometricsense.blogspot.in/2011/11/gradient-descent-in-r.html

Start at some x value, use derivative at that value to tell us which way to move, and repeat. Gradient descent.

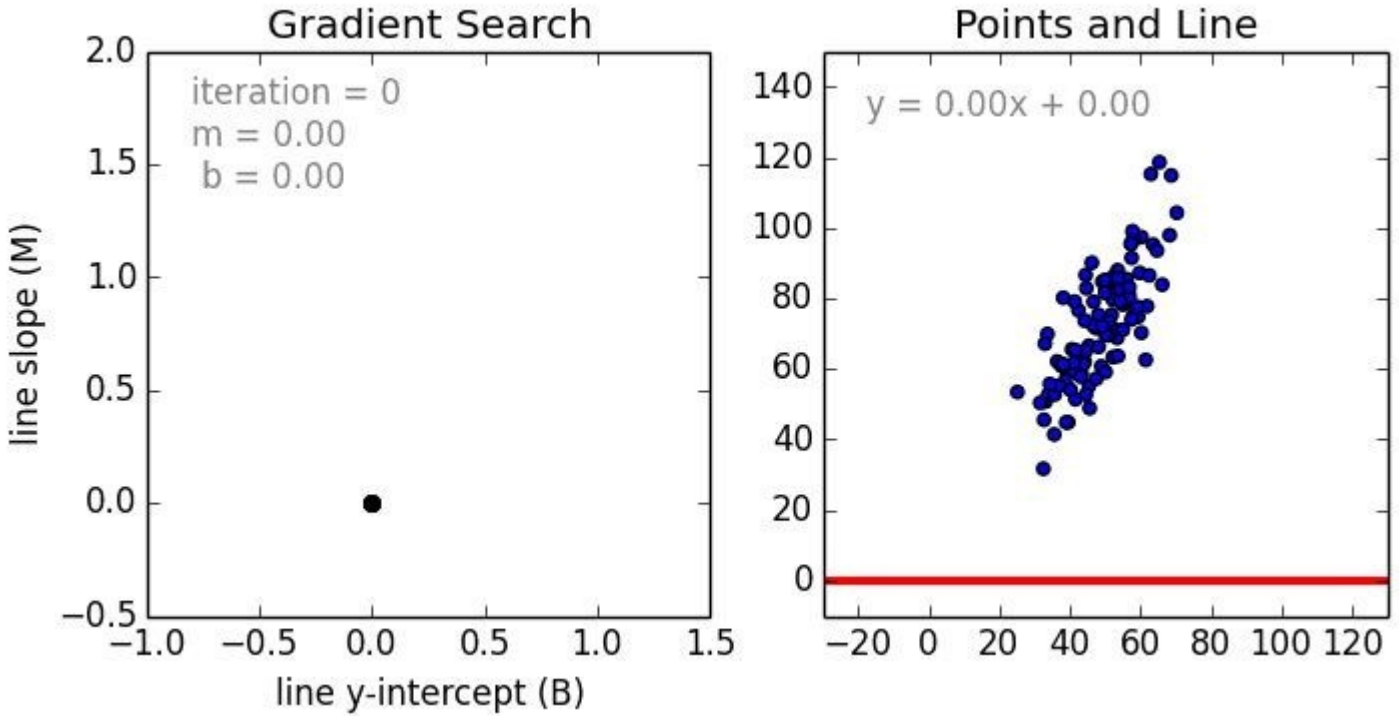
http://www.cs.colostate.edu/%7Eanderson/cs545/Lectures/week6day2/week6day2.pdf



Gradient Descent

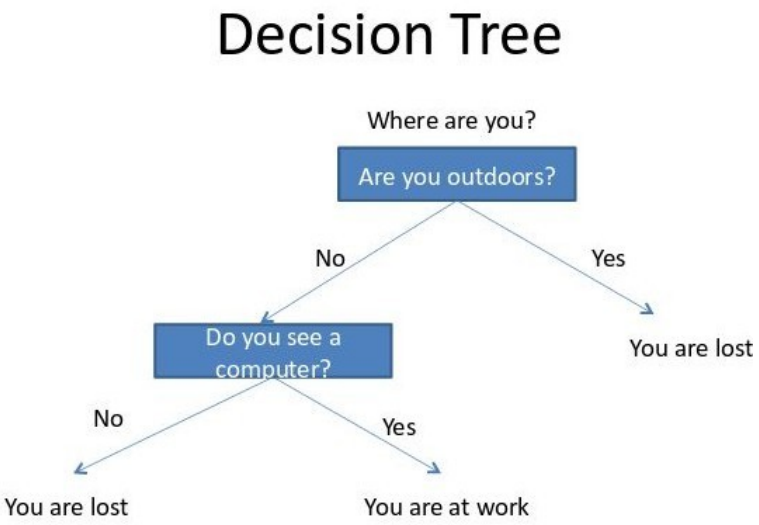
https://spin.atomicobject.com/2014/(
A standard approach to
solving this type of
problem is to define
an error function (also
called a cost function)
that measures how “good”
a given line is.

```
initial_b = 0 # initial y-intercept gues
initial_m = 0 # initial slope guess
num_iterations = 1000
```



Decision Trees

http://select.cs.cmu.edu/class/10701-F09/recit



Decision Trees

Http://www.ise.bgu.ac.il/faculty/liorr/hbchap9.pdf

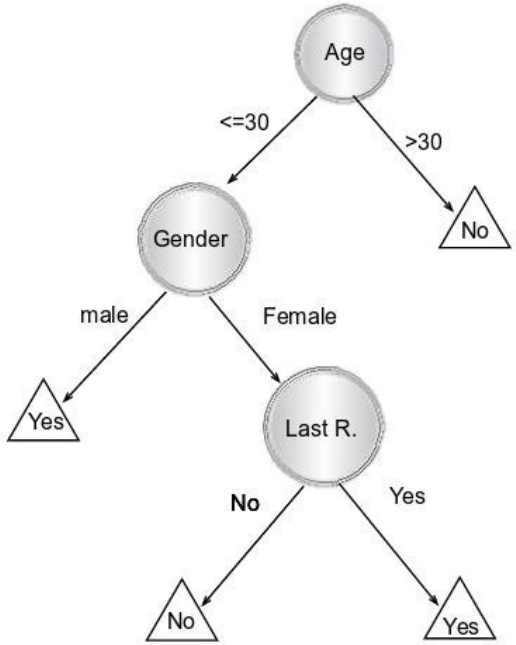


Figure 9.1. Decision Tree Presenting Response to Direct Mailing.

Random Forest

Random Forests grows many classification trees. To classify a new object from an input vector, put the input vector down each of the trees in the forest. Each tree gives a classification, and we say the tree "votes" for that class. The forest chooses the classification having the most votes (over all the trees in the forest).

Each tree is grown as follows:

1. If the number of cases in the training set is N, sample N cases at random - but *with replacement*, from the original data. This sample will be the training set for growing the tree.
2. If there are M input variables, a number m<<M is specified such that at each node, m variables are selected at random out

   of the M and the best split on these m is used to split the node. The value of m is held constant during the forest growing.
3. Each tree is grown to the largest extent possible. There is no pruning.

In the original paper on random forests, it was shown that the forest error rate depends on two things:

- The *correlation* between any two trees in the forest. Increasing the correlation increases the forest error rate.
- The *strength* of each individual tree in the forest. A tree with a low error rate is a strong classifier. Increasing the strength of the individual trees decreases the forest error rate.

https://www.stat.berkeley.edu/~breiman/RandomForests/cc_home.htm#intro

Bagging

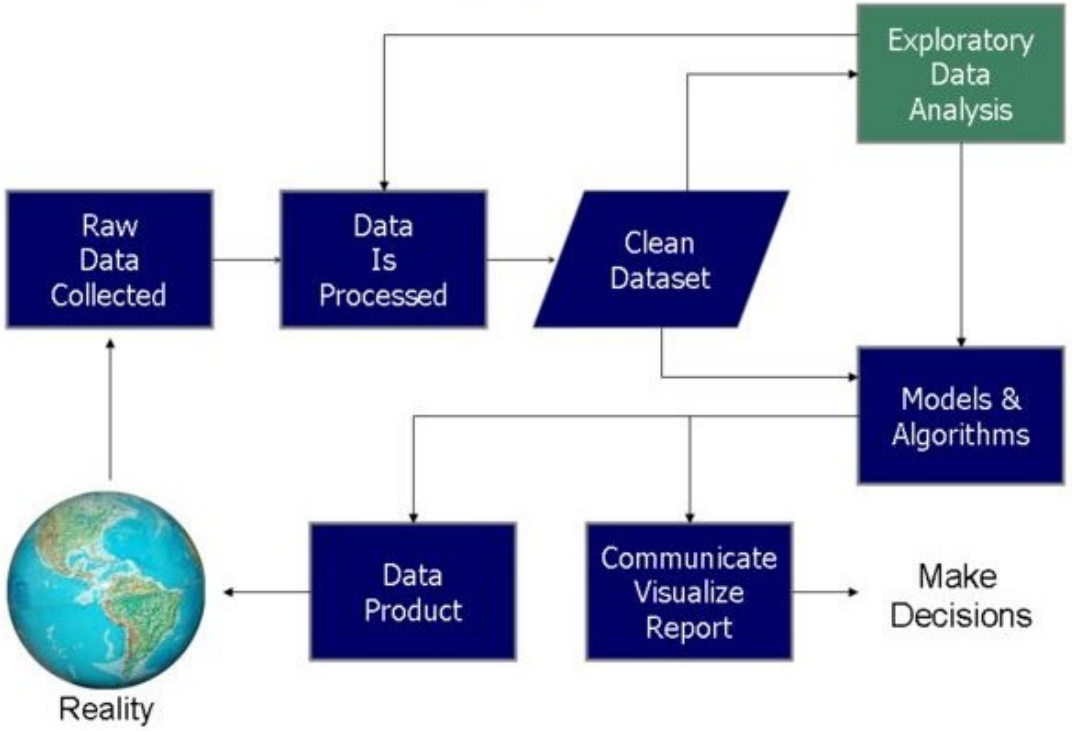
Bagging, aka bootstrap aggregation, is a relatively simple way to increase the power of a predictive statistical model by taking multiple random samples(with replacement) from your training data set, and using each of these samples to construct a separate model and separate predictions for your test set. These predictions are then averaged to create a, hopefully more accurate, final prediction value.

http://www.vikparuchuri.com/blog/build-your-own-bagging-function-in-r/

Boosting is one of several classic methods for creating ensemble models, along with bagging, random forests, and so forth. *Boosting* means that each tree is dependent on prior trees, and learns by fitting the residual of the trees that preceded it. Thus, boosting in a decision tree ensemble tends to improve accuracy with some small risk of less coverage.
XGBoost is a library designed and optimized for boosting trees algorithms. XGBoost is used in more than half of the winning solutions in machine learning challenges hosted at Kaggle.

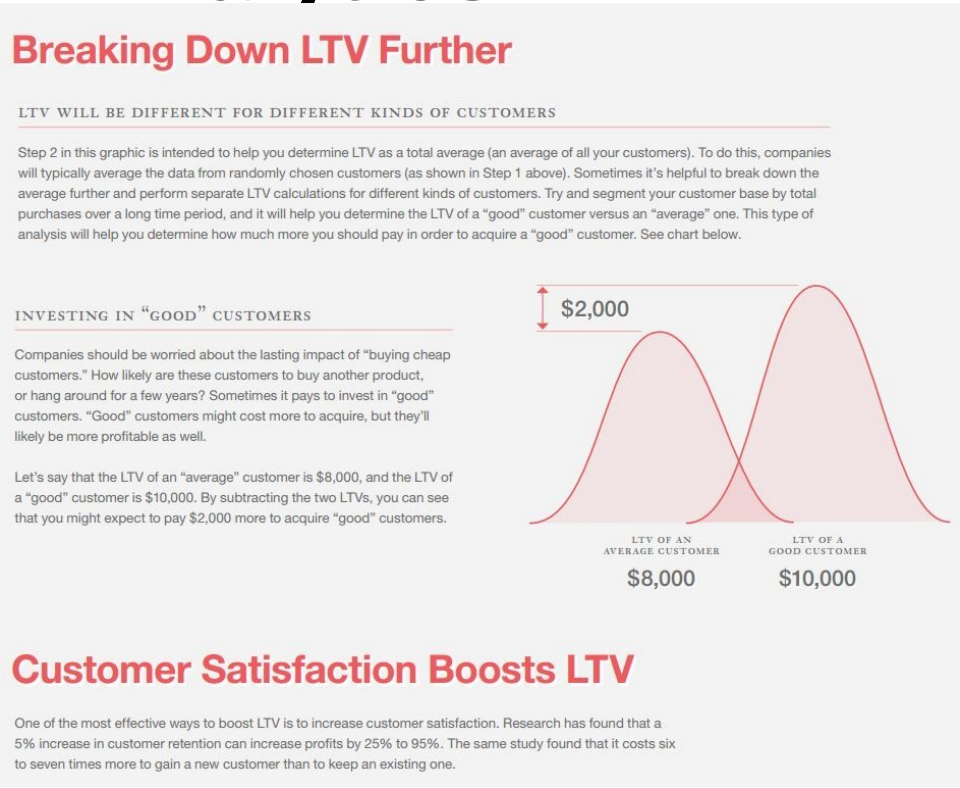
http://xgboost.readthedocs.io/en/latest/model.html#
And http://dmlc.ml/rstats/2016/03/10/xgboost.html

Data Science Process



By Farcaster at English Wikipedia, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=40129394

LTV Analytics



Breaking Down LTV Further

LTV WILL BE DIFFERENT FOR DIFFERENT KINDS OF CUSTOMERS

Step 2 in this graphic is intended to help you determine LTV as a total average (an average of all your customers). To do this, companies will typically average the data from randomly chosen customers (as shown in Step 1 above). Sometimes it's helpful to break down the average further and perform separate LTV calculations for different kinds of customers. Try and segment your customer base by total purchases over a long time period, and it will help you determine the LTV of a "good" customer versus an "average" one. This type of analysis will help you determine how much more you should pay in order to acquire a "good" customer. See chart below.

INVESTING IN "GOOD" CUSTOMERS

Companies should be worried about the lasting impact of "buying cheap customers." How likely are these customers to buy another product, or hang around for a few years? Sometimes it pays to invest in "good" customers. "Good" customers might cost more to acquire, but they'll likely be more profitable as well.

Let's say that the LTV of an "average" customer is $8,000, and the LTV of a "good" customer is $10,000. By subtracting the two LTVs, you can see that you might expect to pay $2,000 more to acquire "good" customers.

Customer Satisfaction Boosts LTV

One of the most effective ways to boost LTV is to increase customer satisfaction. Research has found that a 5% increase in customer retention can increase profits by 25% to 95%. The same study found that it costs six to seven times more to gain a new customer than to keep an existing one.

Life Time Value (LTV) will help us answer 3 fundamental questions:

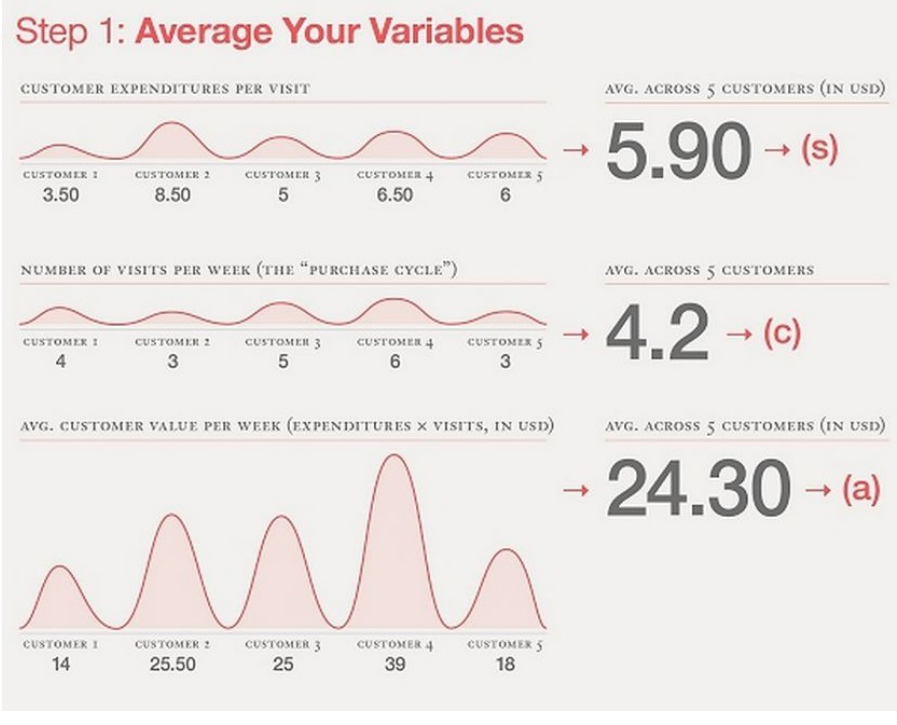
1. Did you pay enough to acquire customers from each marketing channel?

2. Did you acquire the best kind of customers?

3. How much could you spend on keeping them sweet with email and social media?

LTV Analytics :Case Study

https://blog.kissmetrics.com/how-to-calculate-lifetime-value/



LTV Analytics

https://blog.kissmetrics.com/how-to-calculate-lifetime-value/



Step 2: Calculate Lifetime Value (LTV)

CONSTANTS

t **The Average Customer Lifespan** (how long someone remains a customer). In the case of Starbucks, the average customer lifespan is **20 years**.

r **Customer Retention Rate.** The percentage of customers, who, over a given period of time, repurchase, when compared to an equal and preceding period of time. Starbucks: **75%**.

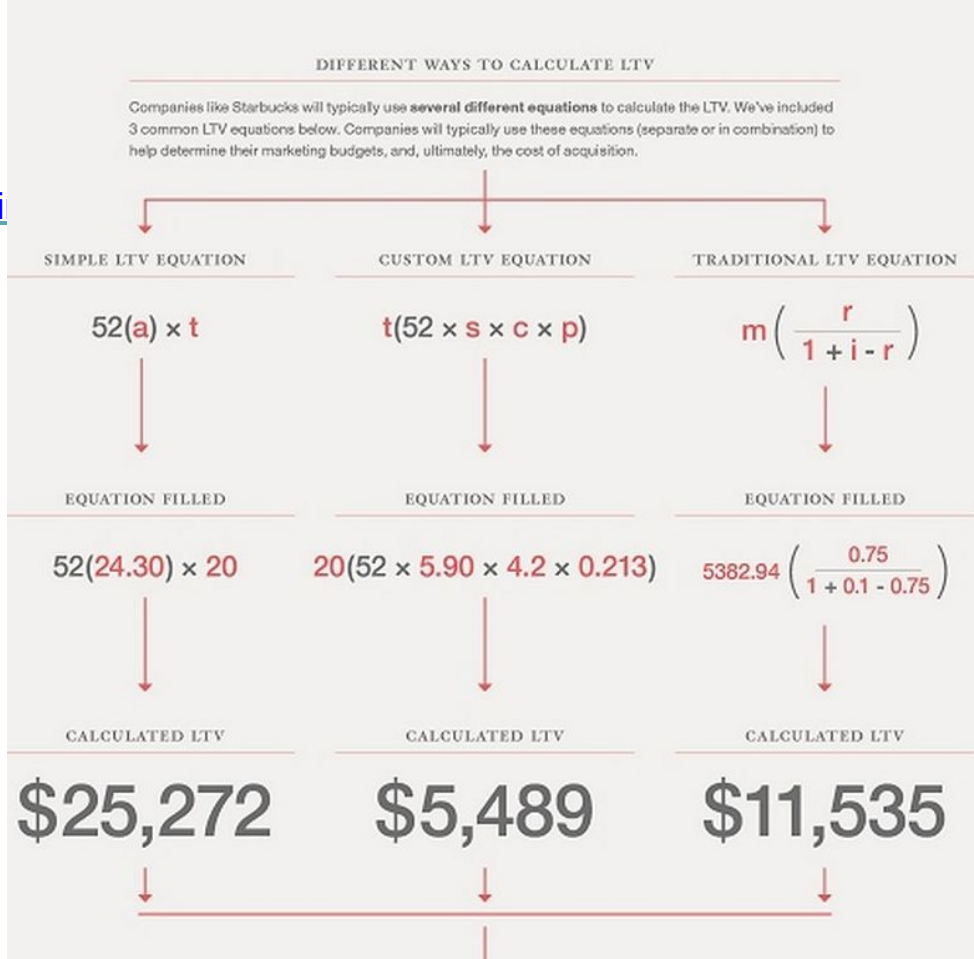
p **Profit Margin per Customer.** Starbucks: **21.3%**.

i **The Rate of Discount.** The "rate of discount" is the interest rate used in discounted cash flow analysis to determine the present value of future cash flows. Usually this number falls between 8% and 15%. Starbucks: **10%**.

m **Avg. Gross Margin per Customer Lifespan.** Starbucks has a profit margin of 21.3% (see constant "p"). If the average customer spends $25,272 (see the "Simple LTV Equation" results below) during their time as a customer ("t"), Starbucks has gross margin per customer lifespan of **$5382.94**.

LTV Analytics

https://blog.kissmetrics.com/how-to-calculate-lifeti



LTV Analytics

https://blog.kissmetrics.com/how-to-calculate-lifetime-value/



LTV Analytics

http://www.kaushik.net/avinash/analytics-tip-calculate-ltv-customer-lifetime-value/

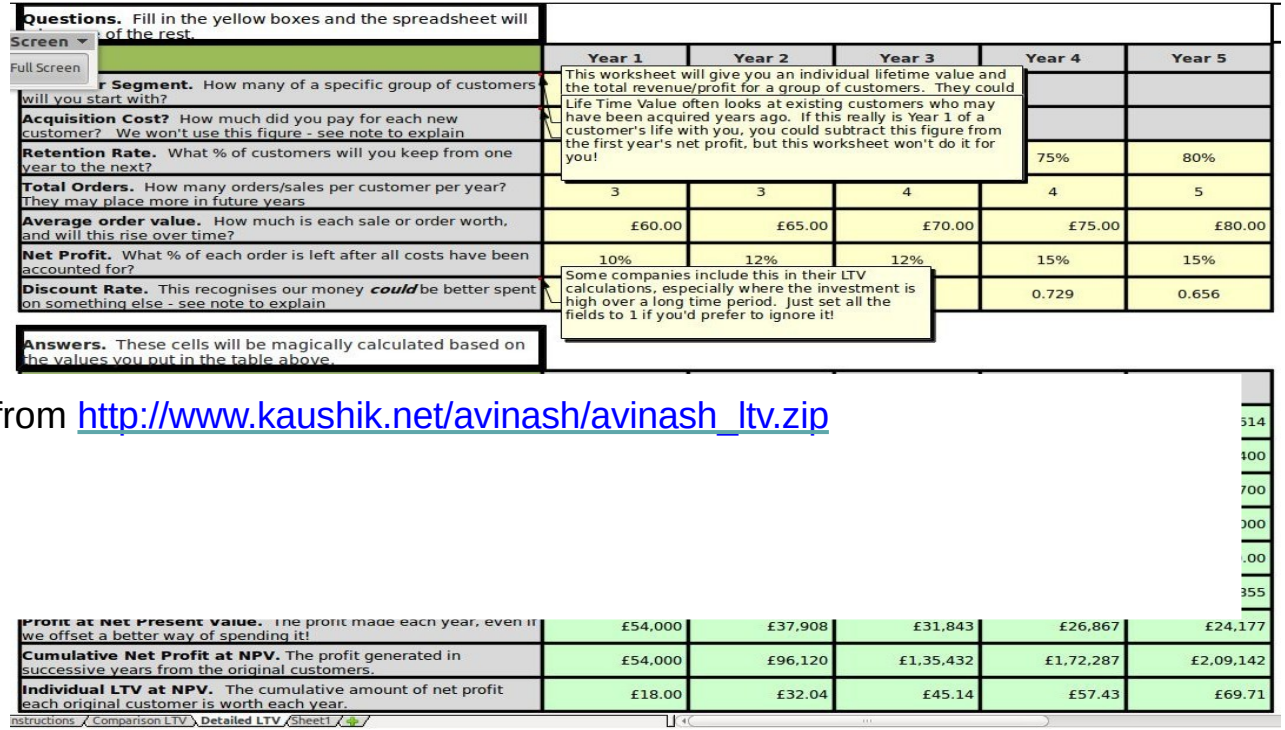
Questions. Fill in the yellow boxes and the spreadsheet will take care of the rest.

	Best Customers	Average Customers
Acquisition Cost .How much did you pay to acquire these customers?	£40.00	£12.00
Average order value. How much do they spend per order?	£92.00	£70.00
Orders per year? Quite simply, How many orders do they place per year?	5	2
Retention? How many years will they be customers for?	3	2
Net profit? What is the net profit percentage of goods sold?	10%	10%

Answers. These cells will be magically calculated based on the values you put in the table on the left

	Best Customers	Average Customers
Lifetime Gross Revenue	£1,380.00	£280.00
Life Time Net Profit	£98.00	£16.00

LTV



Download the zip file from http://www.kaushik.net/avinash/avinash_ltv.zip

Pareto principle

The **Pareto principle** (also known as the **80–20 rule**, the **law of the vital few,** and the **principle of factor sparsity**) states that, for many events, roughly 80% of the effects come from 20% of the causes

- 80% of a company's profits come from 20% of its customers
- 80% of a company's complaints come from 20% of its customers
- 80% of a company's profits come from 20% of the time its staff spend
- 80% of a company's sales come from 20% of its products
- 80% of a company's sales are made by 20% of its sales staff

Several criminology studies have found 80% of crimes are committed by 20% of criminals.

RFM Analysis

RFM is a method used for analyzing customer value.

- **R**ecency - *How recently did the customer purchase?*
- **F**requency - *How often do they purchase?*
- **M**onetary Value - *How much do they spend?*

A method

- Recency = 10 - the number of months that have passed since the customer last purchased
- Frequency = number of purchases in the last 12 months (maximum of 10)
- Monetary = value of the highest order from a given customer (benchmarked against $10k)

Alternatively, one can create categories for each attribute. For instance, the Recency attribute might be broken into three categories: customers with purchases within the last 90 days; between 91 and 365 days; and longer than 365 days. Such categories may be arrived at by applying business rules, or using a data mining technique, to find meaningful **breaks**.

A commonly used shortcut is to use deciles. One is advised to look at distribution of data before choosing breaks.

Are you ready
To use more
Data Science

